

AURA

Academic Unified Response & Assistance

Software Engineering Project

Milestone 6 Final Report

Presented By:

Aryan Kumar (22f2000697)

Aziz Ahmed (21f2000480)

Imran Ashraf (22f1001972)

Laxmi Kumari (21f1002304)

Mohit Raj Rathor (22f3003109)

Nooha Rahman C.P (21f1006476)

Taniya Chouhan (23f2003584)

Vikas Rathore (22f1001805)

Table of Contents

1. About the Project	03
2. Milestone 1	04
3. Milestone 2	10
4. Milestone 3	26
5. Milestone 4	48
6. Milestone 5	57
7. Issue Reporting and Tracking	65
8. Technologies used	68
9. Instruction to Run the Application	70

ABOUT THE PROJECT

This project aims to build an AI-powered support system that simplifies learning and communication across the IITM academic ecosystem. It brings together features such as instant multilingual query resolution, smart study summaries, personalized resource recommendations, doubt and discussion summarization, and automated query routing. The system also provides tools for instructors and TAs, including engagement insights, onboarding assistance, assessment generation, and slide creation.

Overall, the project reduces repetitive work, helps students stay organized, and gives instructors clearer visibility into student needs—creating a more efficient and supportive learning experience for everyone.

THE TEAM

Role	Member
Project Manager / Architecture	Mohit Raj Rathor
Scrum Master	Aryan Kumar
Frontend Developer	Vikas Rathore
	Taniya Chouhan
Backend Developer	Imran Ashraf
	Nooha Rehman C.P.
	Laxmi Kumari
	Aziz Ahmed

Milestone 1

IDENTIFYING USER REQUIREMENTS

IDENTIFIED USER

Primary Users

Teaching Assistants (TAs) and Instructors are the core users of our system. The platform is built mainly to support their academic responsibilities by reducing repetitive tasks, improving communication, and ensuring smoother student query management.

TAs: Use the system to track, summarize, and manage student queries efficiently. They can monitor unresolved doubts, handle escalations, and get quick summaries to prepare for sessions.

Instructors: Use AI-generated analytics, summaries, and automation tools like question paper and slide generators to save time and focus on teaching quality and student engagement.

They are the primary users because the solution directly enhances their efficiency and teaching outcomes.

Secondary Users

Students are the secondary users.

They interact frequently with the platform through the AI assistant and discussion forums but mostly experience its impact through the improved responsiveness of TAs and instructors. Use the AI assistant for academic and administrative questions. Get weekly summaries, reminders, and personalized content recommendations. Participate in course-level discussions and collaborative spaces moderated by AI.

While they are the largest user group, the system is designed primarily to support their learning through better instructor and TA engagement.

IDENTIFIED USER

Tertiary Users

Administrators and **Developers** are the tertiary users.

They do not use the system for learning or teaching but play a key role in maintaining and improving it.

Administrators: Use analytics dashboards to track query trends, update FAQs, and ensure information accuracy.

Developers/Support Team: Handle integration, updates, and model optimization to keep the AI system accurate and efficient.

Their role is essential for smooth operations and long-term reliability of the system.

USER STORIES

As a **TA**, I want repetitive student queries to be **auto-answered accurately**, so that I can **focus more on academic assistance** than on clerical responses.

As a **student**, I want to get **accurate answers to academic and administrative questions instantly**, so that I **don't waste time** searching through multiple portals or PDFs.

As an **instructor**, I want **insights into student engagement patterns**, so that I can **plan interactive sessions and maintain motivation** throughout the term.

As a **student**, I want **automated weekly summaries** of lectures, announcements, and tasks, so that I can **stay organized** without manually tracking every update.

As a **TA**, I want a **curated list of high-quality learning tools and references**, so that I can **quickly guide students** instead of spending hours researching.

As a **student**, I want **personalized resource suggestions** related to IITM lecture topics, so that I can **easily understand complex subjects** without endless searching.

As an **instructor**, I want **summarized reports of confusing topics**, so that I can **refine lecture delivery and provide targeted clarifications**.

As a **TA**, I want frequent student **doubts automatically summarized** from chat forums and emails, so that I can identify and **address common learning issues efficiently**.

As a **TA**, I want unresolved or delayed **queries to be tracked and prioritized**, so that I can **manage my workload and response time effectively**.

As a **student**, I want my queries to **automatically reach the right instructor or TA**, so that I get **timely and accurate responses without confusion**.

USER STORIES

As an **instructor**, I want **summarized discussion threads grouped by topic**, so that I can **easily review student challenges before class**.

As a **student**, I want to **quickly find if my question has been discussed earlier**, so that I **don't waste time posting duplicates**.

As a **new TA**, I want a **personalized onboarding guide based on senior TAs' experiences**, so that I can **adapt faster to academic and communication expectations**.

As an **instructor**, I want to auto **generate question papers of varying difficulty with editable templates**, so that I can **save time preparing balanced assessments**.

As a **TA or instructor**, I want to convert structured **course notes into visually consistent slides**, so that I can **create professional teaching material quickly without manual formatting**.

As an **administrator**, I want to **view analytics of frequently asked questions and unresolved queries**, so that I can **identify policy gaps and improve institutional resources**.

As an **instructor**, I want to **see patterns in student confusion and feedback sentiment**, so that I can **adapt my teaching strategies accordingly**.

INTERVIEWS

S. No.	Interviewee	His/her Details	Interviewer	Proof Drive link
1	S Satyanarayana	Course instructor	Nooha Rahman	Link1
2	Pulkit Mangal	Intro to DL and GenAI TA	Aryan Kumar	Link2
3	Abhijit Kumar	Intro to DL and GenAI TA	Aryan Kumar	Link3
4	Saranath P	TA : Data Science and AI Lab	Laxmi Kumari	Link4
5	Sujal Raj Pradhan	Currently working as TDS TA	Mohit Raj Rathor	Link5
6	Md Athar	Student	Nooha Rahman	Link6
7	Bhavya	BS Electronic Student at IITM	Taniya Chouhan	Link7
8	Jhilmil Dutta	Administration intern	Nooha Rahman	Link8
9	Divyansh Gupta	T.A of PDSA	Nooha Rahman	Link9
10	Shramana Sinha	Ex-student	Nooha Rahman	Link10

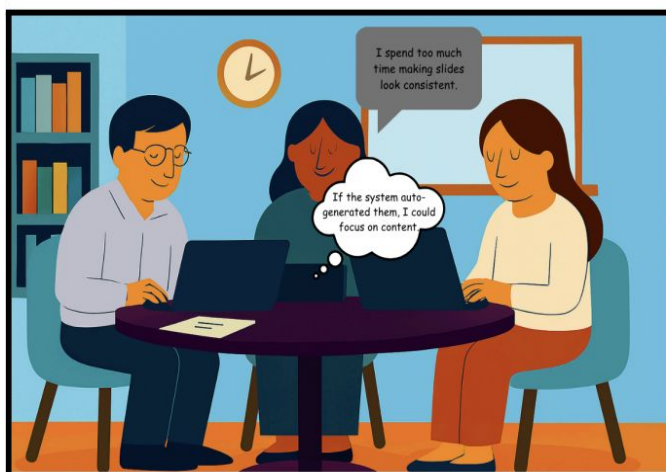
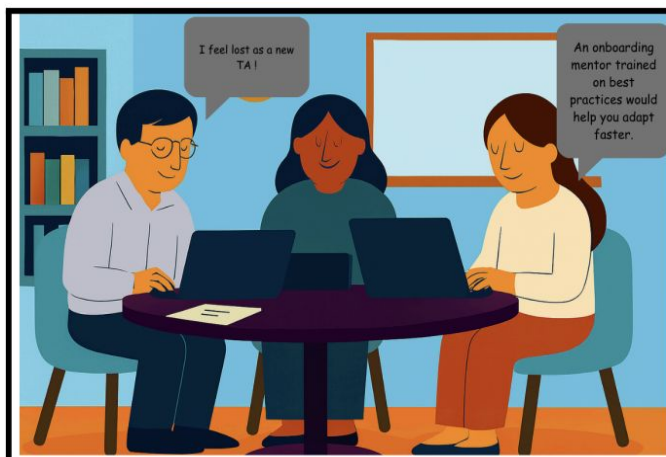
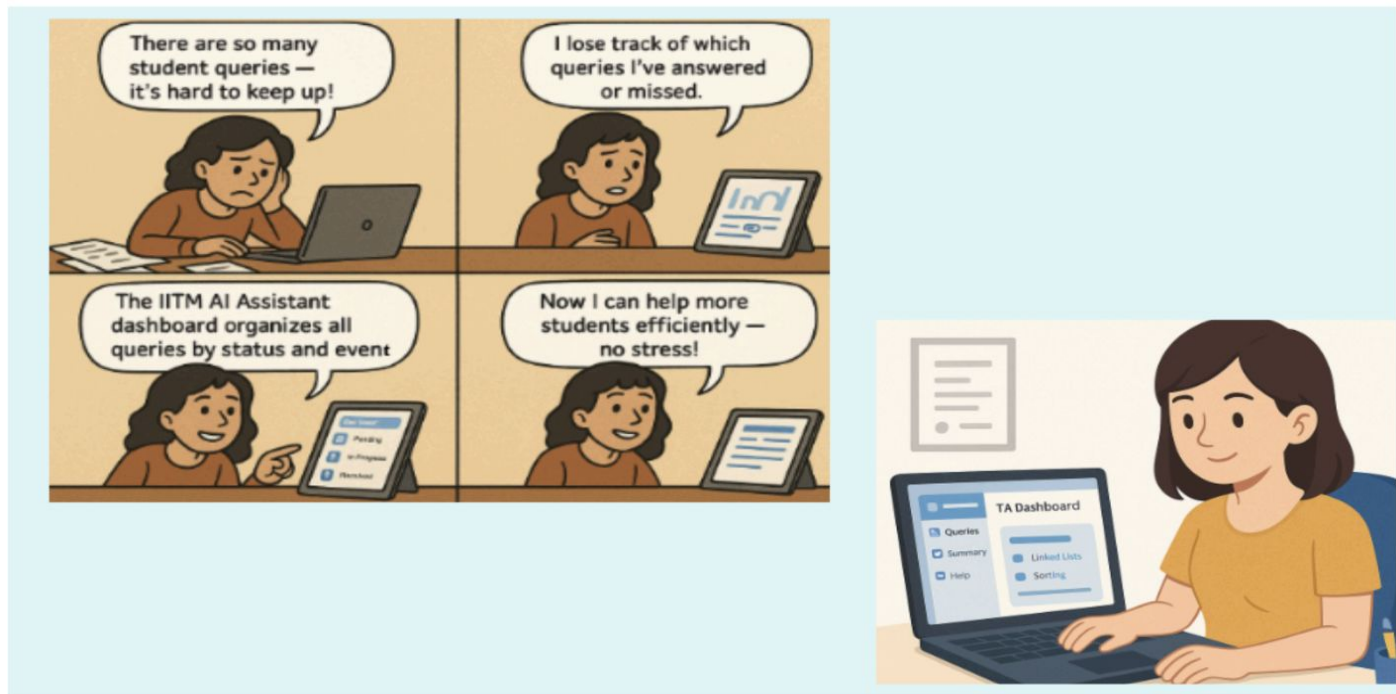
Milestone 2

USER

INTERFACES

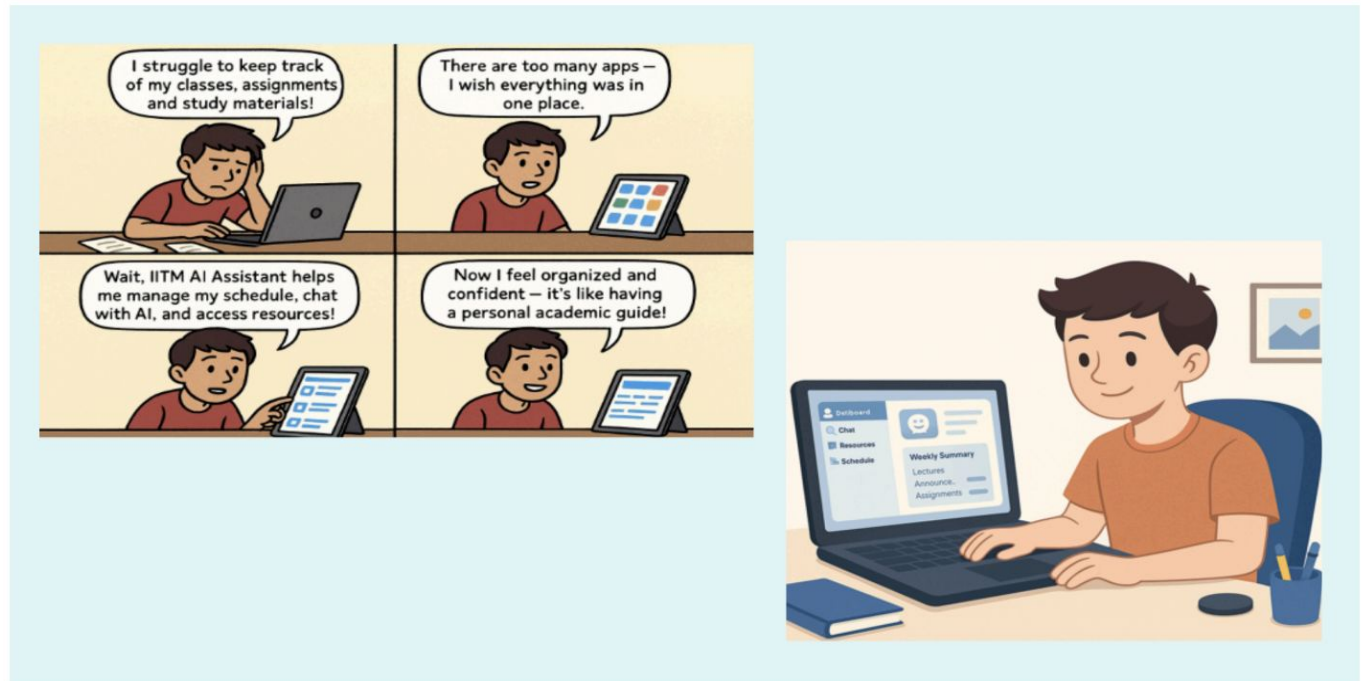
STORYBOARDS

From TA's Perspective



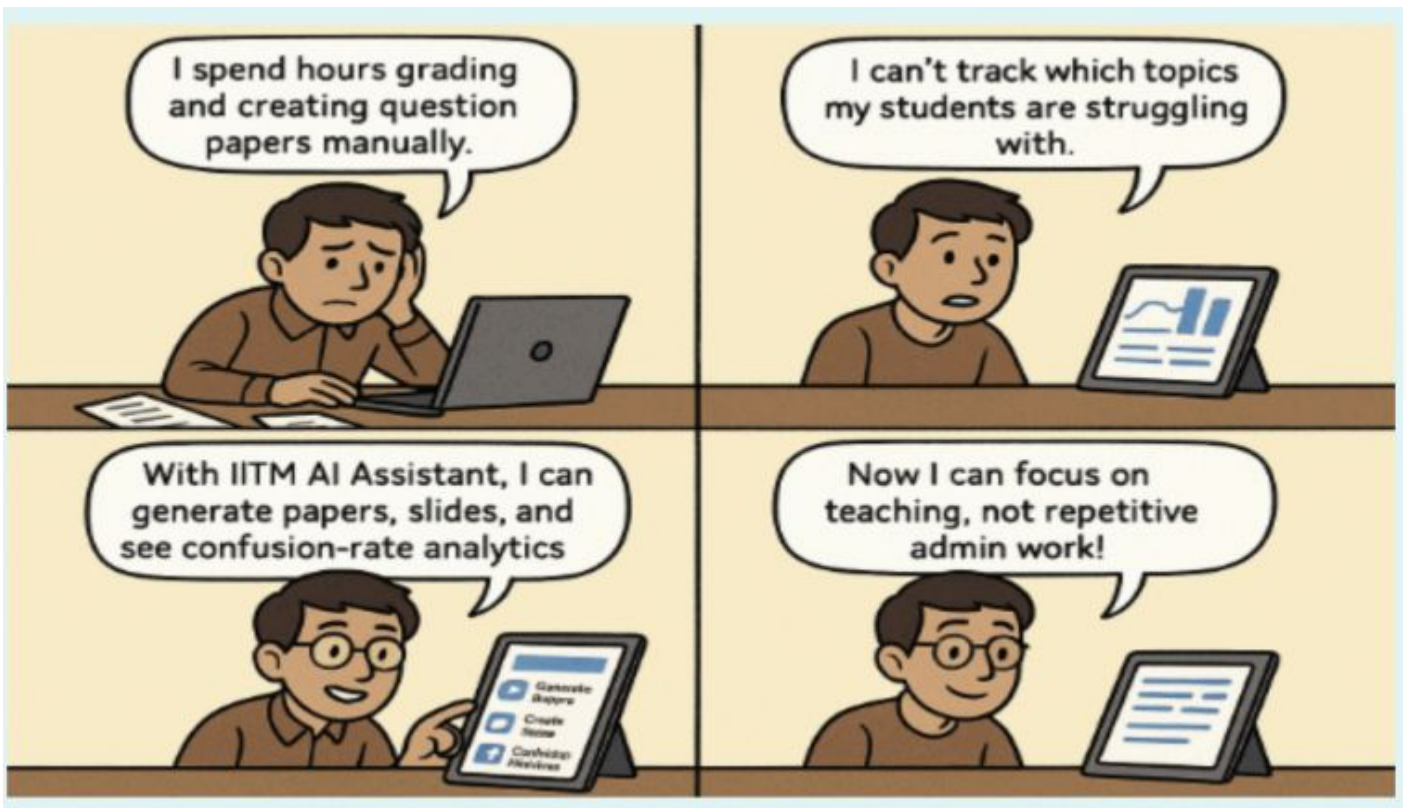
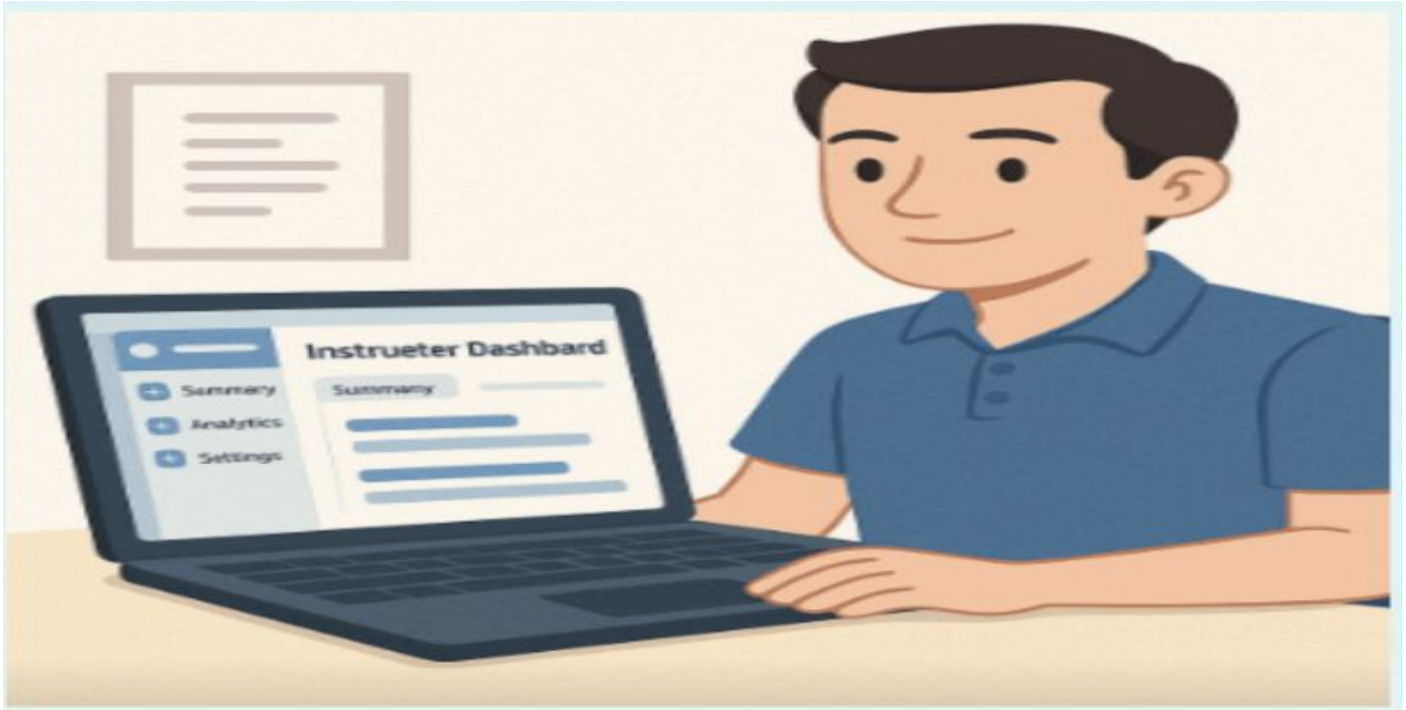
STORYBOARDS

From Student's Perspective



STORYBOARDS

From Instructor's Perspective



STORYBOARDS

From Instructor's Perspective



STORYBOARDS

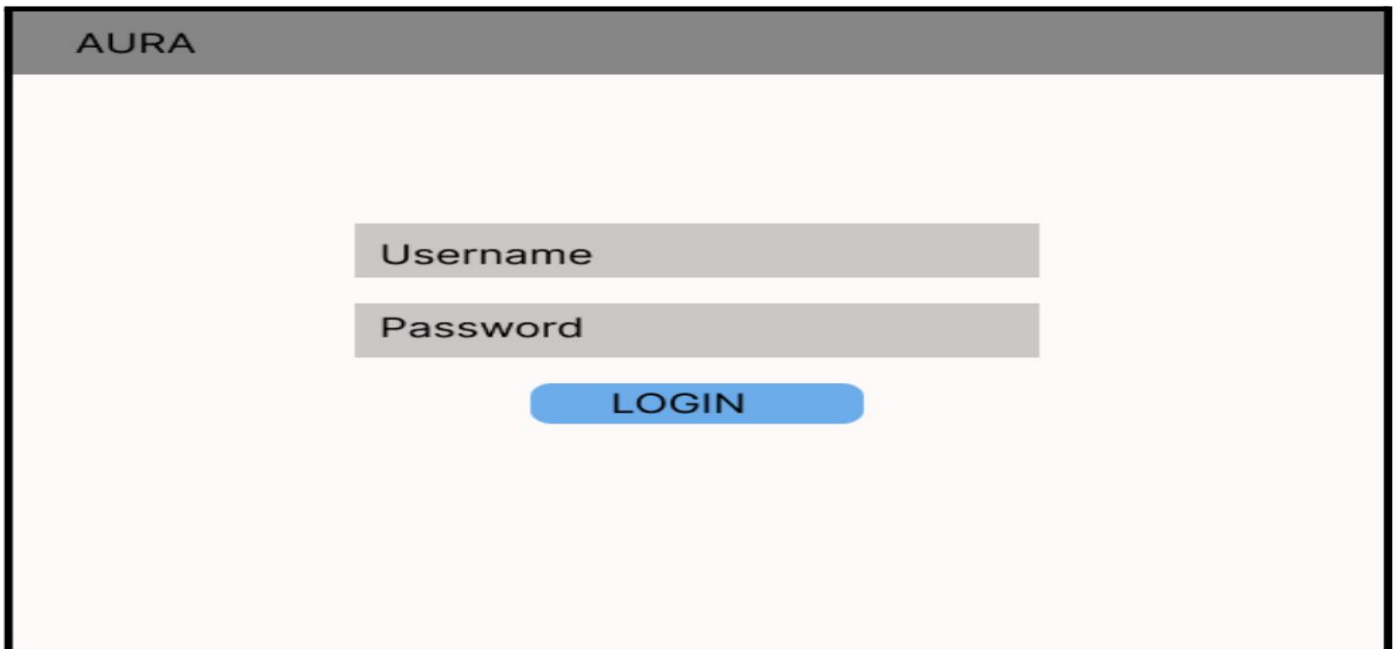
From Admin's Perspective



LOW-FIDELITY WIREFRAMES

BEFORE FEEDBACK

Login page



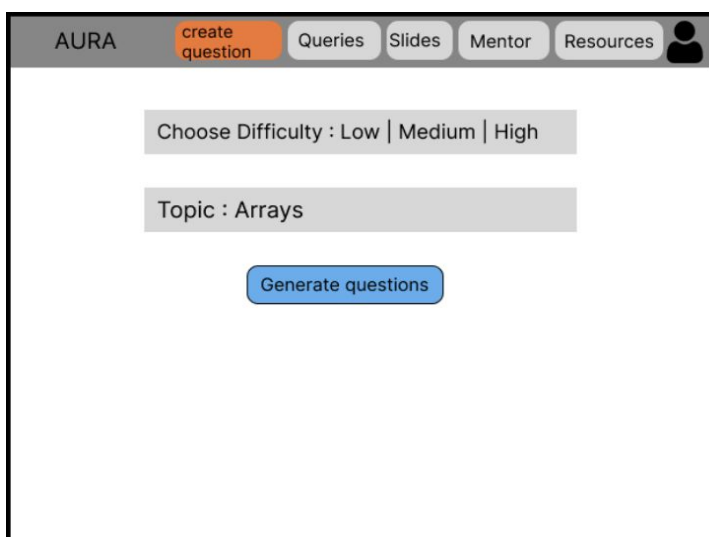
AURA

Username

Password

LOGIN

From TA's Perspective

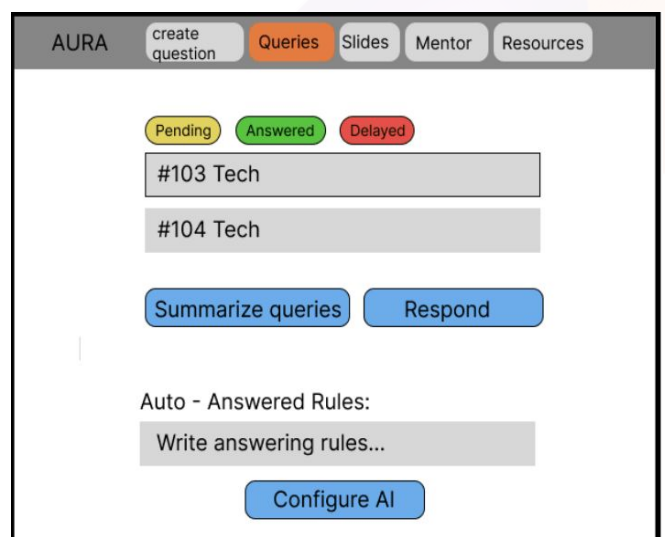


AURA create question Queries Slides Mentor Resources

Choose Difficulty : Low | Medium | High

Topic : Arrays

Generate questions



AURA create question Queries Slides Mentor Resources

Pending Answered Delayed

#103 Tech

#104 Tech

Summarize queries Respond

Auto - Answered Rules:

Write answering rules...

Configure AI

LOW-FIDELITY WIREFRAMES

BEFORE FEEDBACK

From TA's Perspective

AURA

create question

Queries

Slides

Mentor

Resources

Enter the Topic : backpropogation

Content...

Generate Slide

AURA

create questions

Queries

Slides

Mentor

Resources

Welcome, New TA!

AI : Hi! I'm your onboarding mentor.
What do you need help with?

TA: How do I grade assignments?

AI: Here's the 3-step workflow +
rubric template ↓

Ask Senior TA

Contact Faculty

Escalation History:

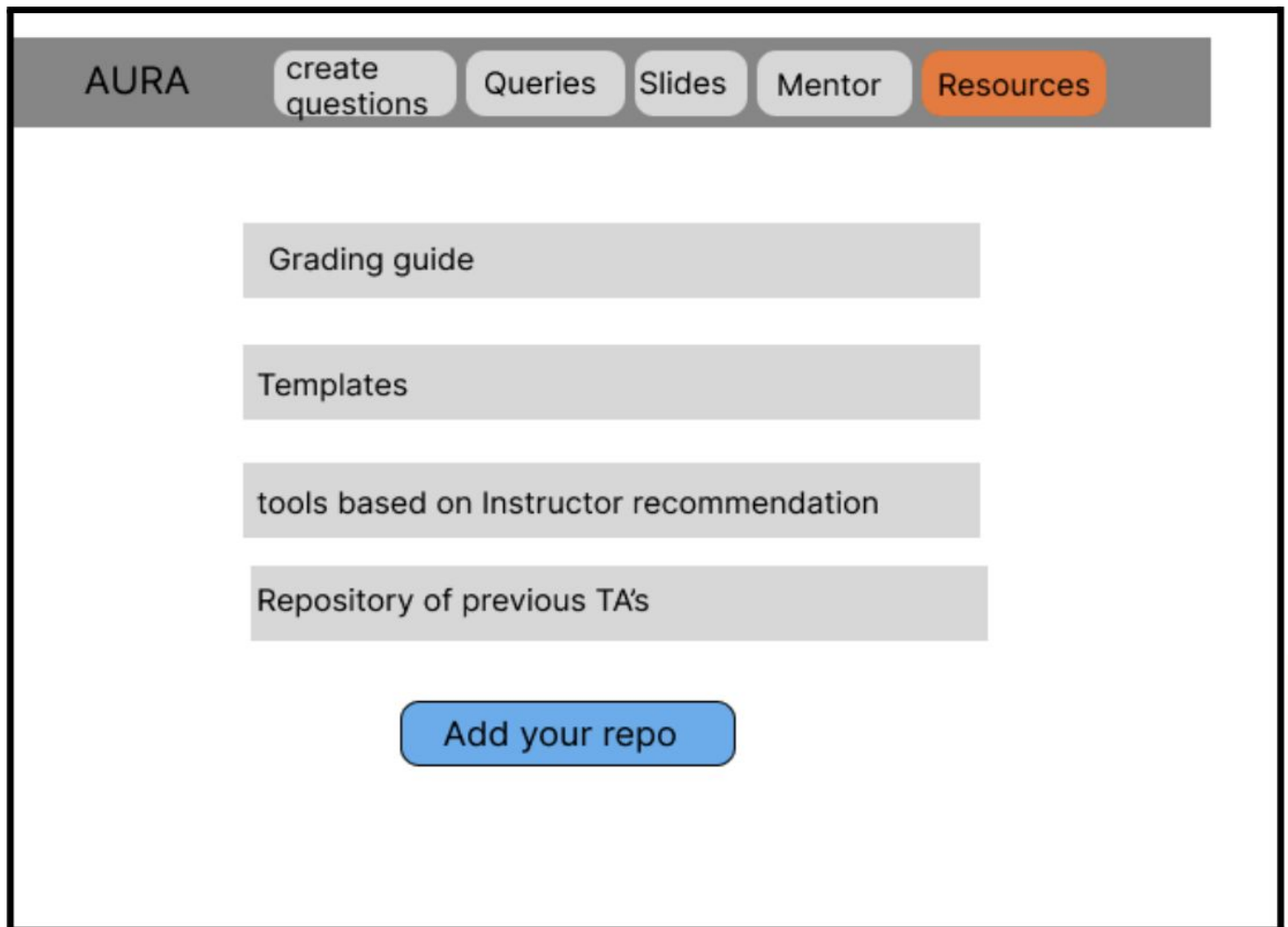
#1: Asked Prof. Rao → Replied in 2h

Complete onboarding

LOW-FIDELITY WIREFRAMES

BEFORE FEEDBACK

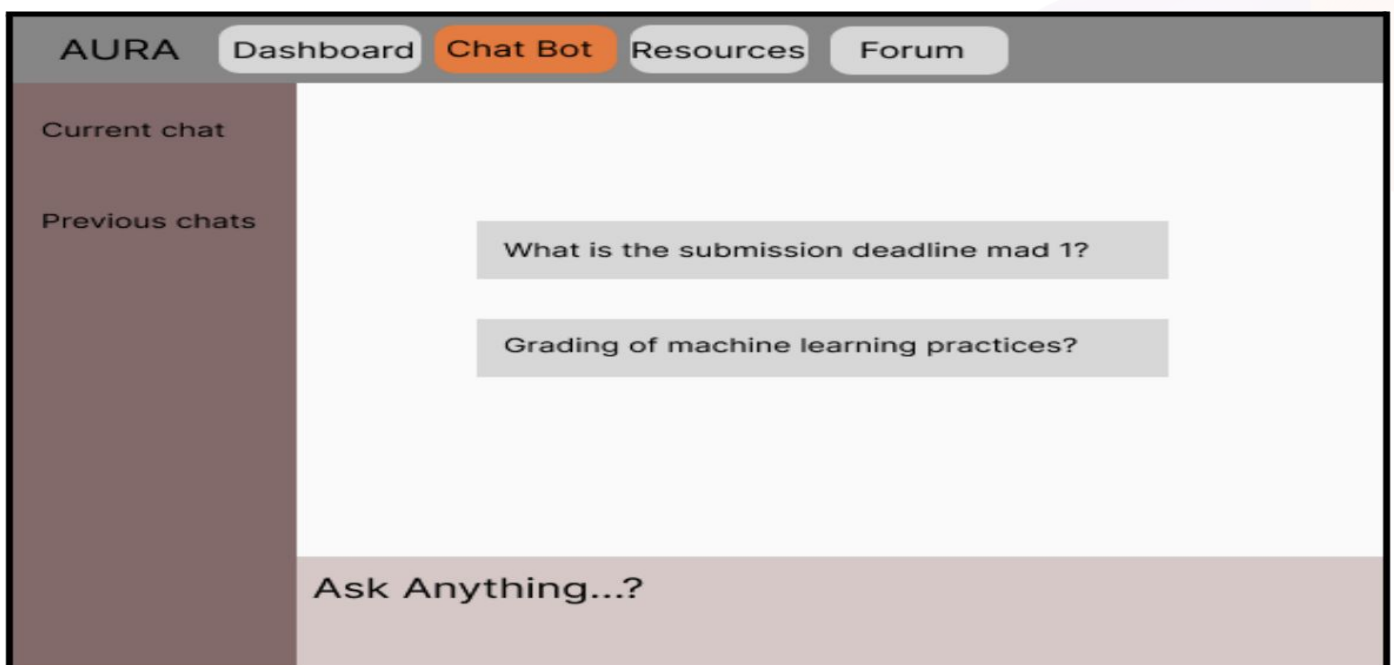
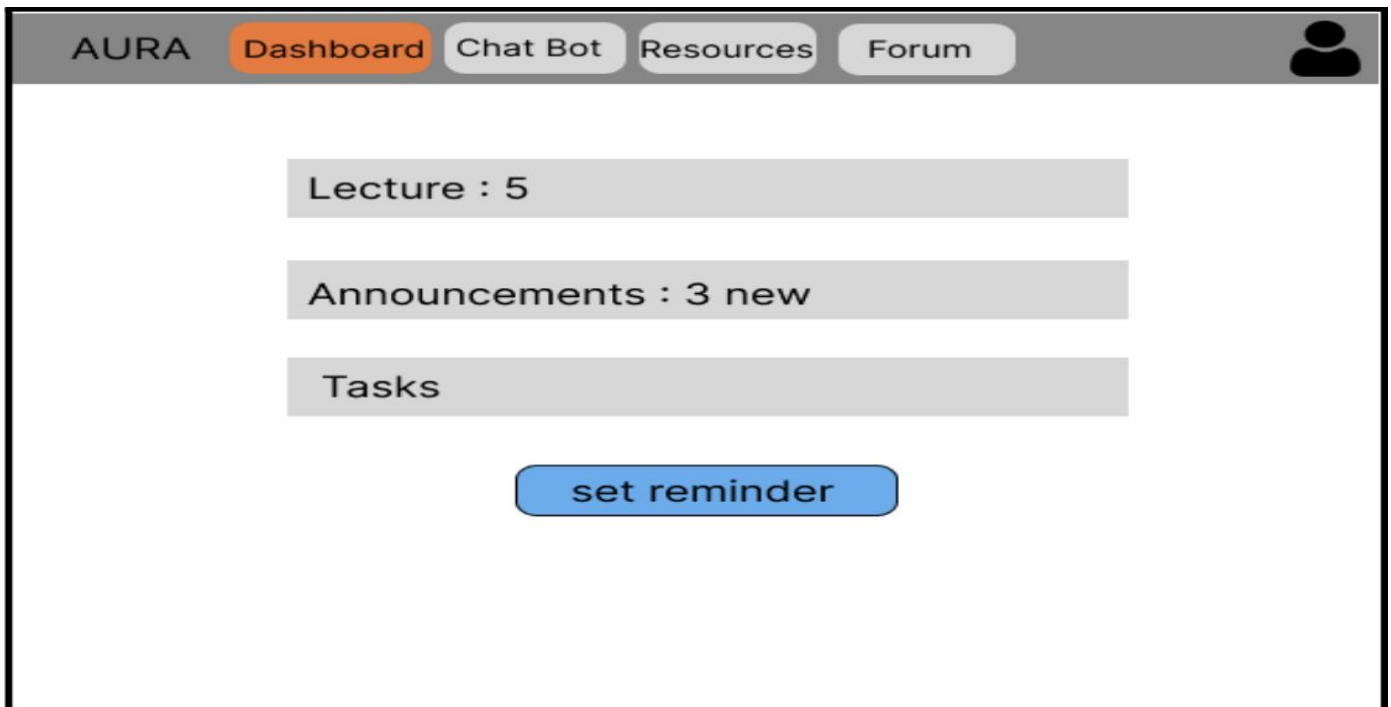
From TA's Perspective



LOW-FIDELITY WIREFRAMES

BEFORE FEEDBACK

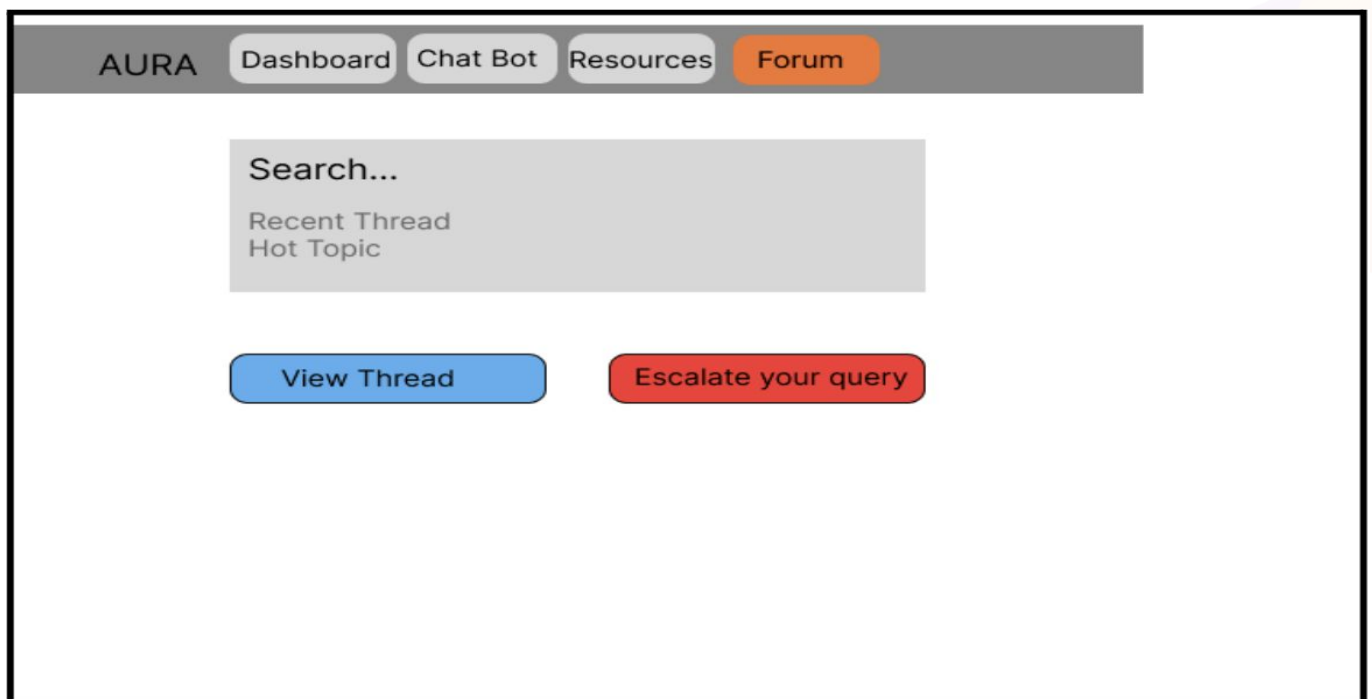
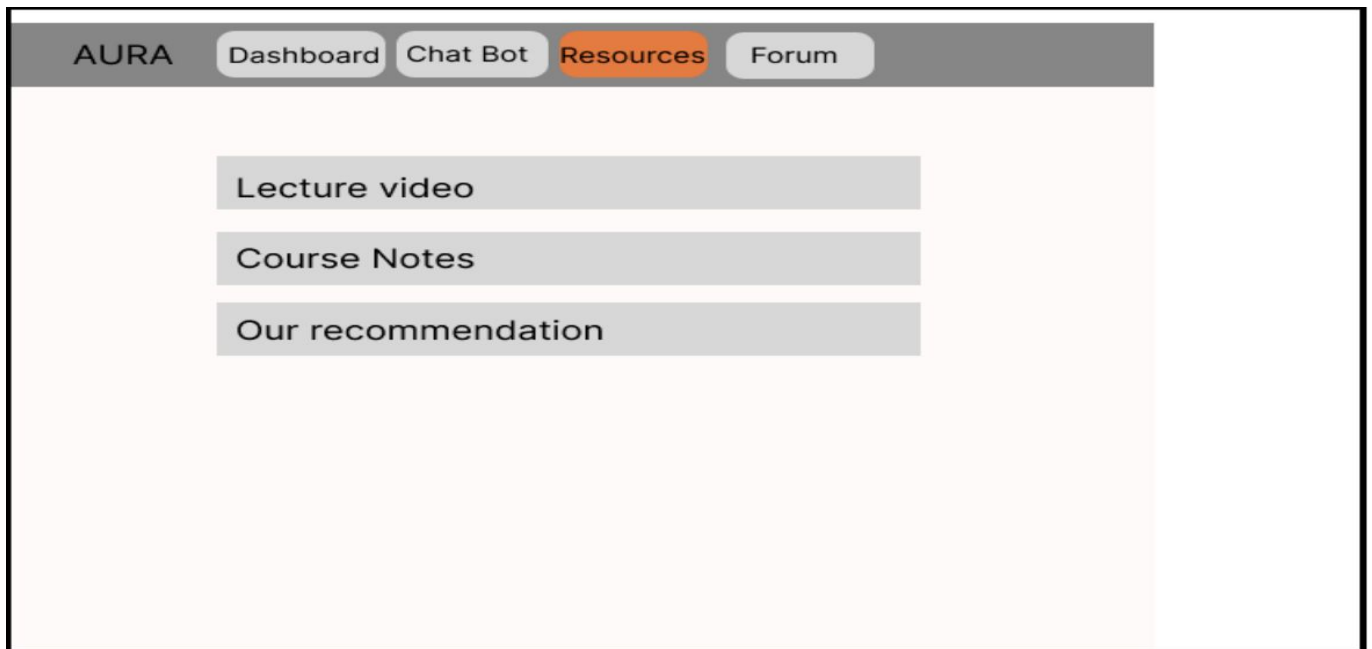
From Student's Perspective



LOW-FIDELITY WIREFRAMES

BEFORE FEEDBACK

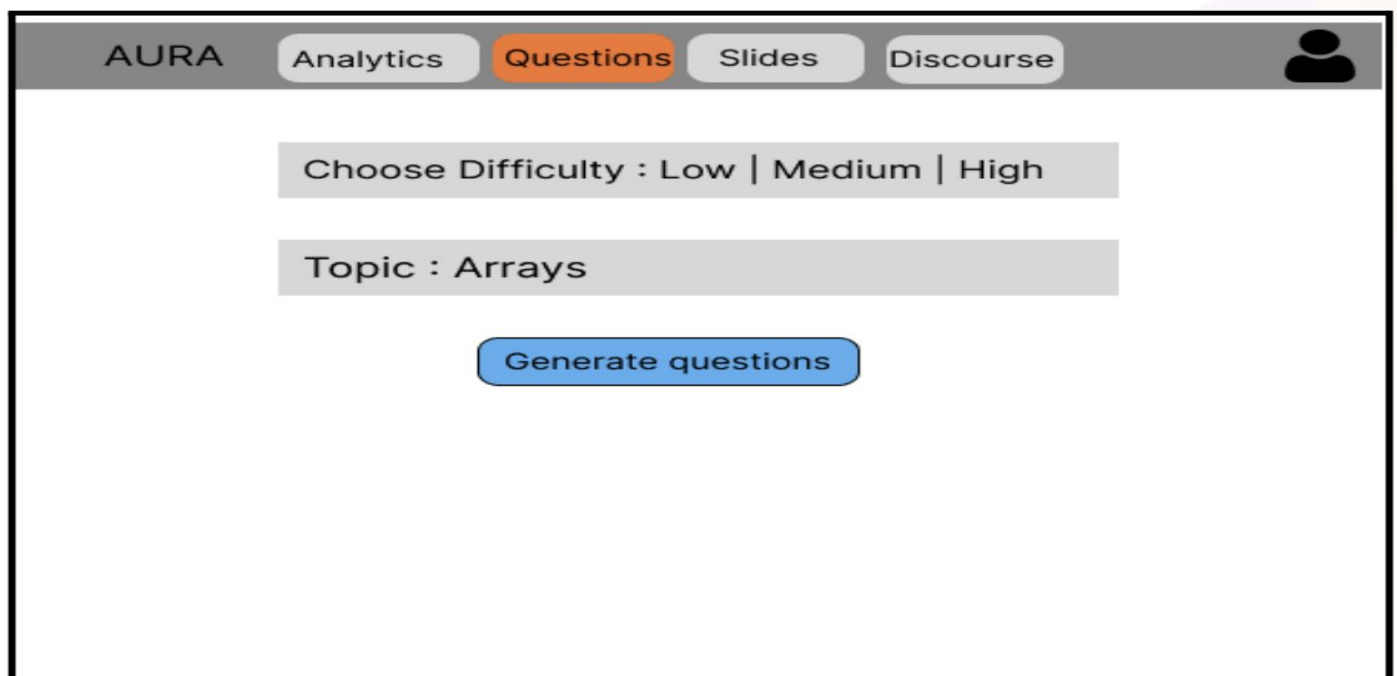
From Student's Perspective



LOW-FIDELITY WIREFRAMES

BEFORE FEEDBACK

From Instructor's Perspective



LOW-FIDELITY WIREFRAMES

BEFORE FEEDBACK

From Instructor's Perspective

AURA Analytics Questions **Slides** Discourse

Enter the Topic : backpropogation

Content...

Generate Slide

This wireframe shows a navigation bar with 'AURA', 'Analytics', 'Questions', 'Slides' (highlighted), and 'Discourse'. Below the bar is a text input field containing 'Enter the Topic : backpropogation', a large gray box labeled 'Content...', and a blue 'Generate Slide' button.

AURA Analytics Questions Slides **Discourse** 

Summarized Discussion Threads

MLP oppe 2

grading policy

MLP Level 2 Viva

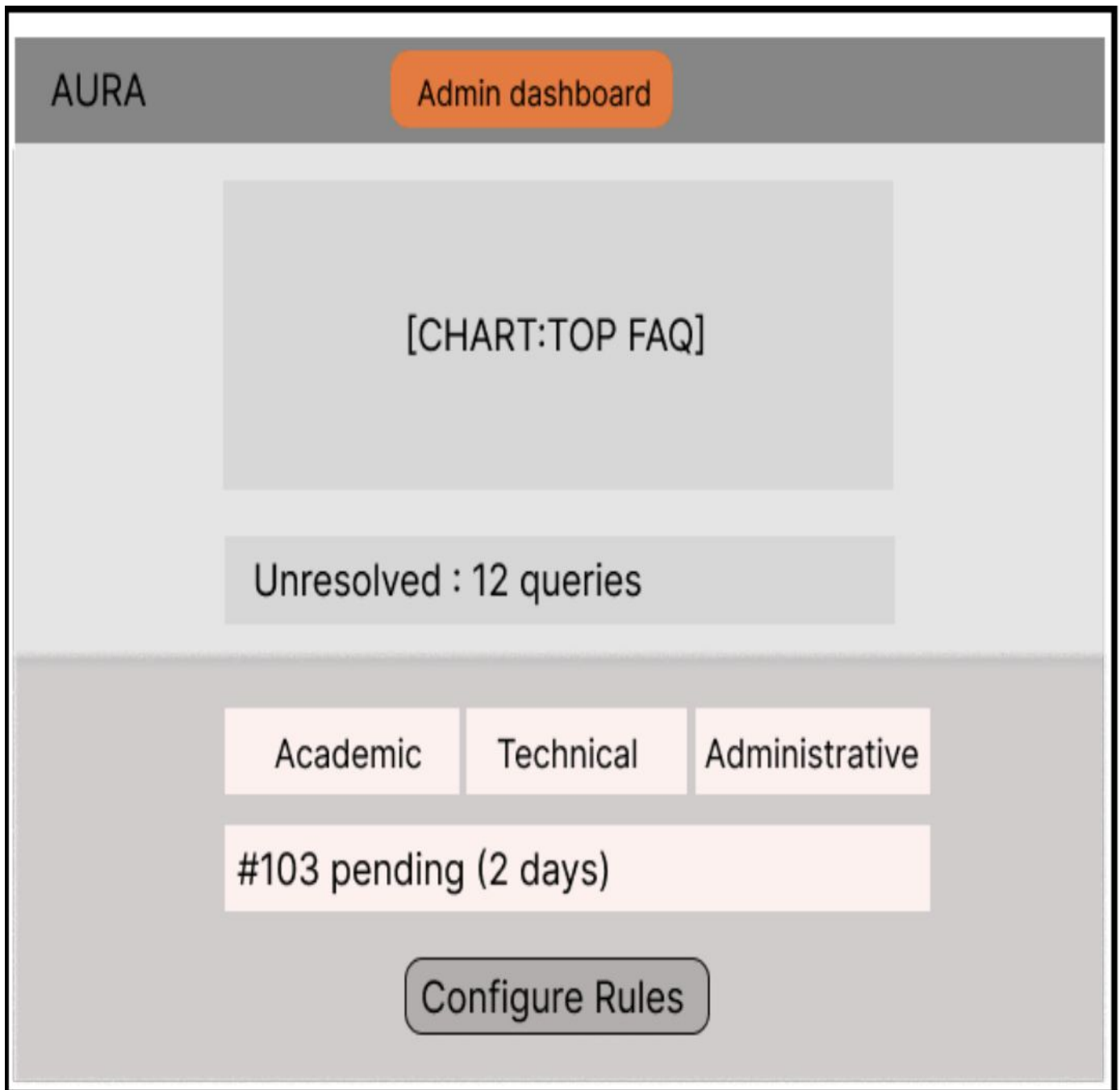
View

This wireframe shows a navigation bar with 'AURA', 'Analytics', 'Questions', 'Slides', 'Discourse' (highlighted), and a user icon. Below the bar is the heading 'Summarized Discussion Threads', followed by three gray boxes containing the text 'MLP oppe 2', 'grading policy', and 'MLP Level 2 Viva'. At the bottom is a blue 'View' button.

LOW-FIDELITY WIREFRAMES

BEFORE FEEDBACK

From Administrator Team's Perspective



LOW-FIDELITY WIREFRAMES

AFTER FEEDBACK

Feedback videos

Video-1 [Click here](#)

Video-2 [Click here](#)

From Instructor's Perspective

The wireframe shows a web interface with a top navigation bar containing the text 'AURA' and four buttons: 'Analytics', 'Questions', 'Slides' (highlighted in orange), and 'Discourse'. Below the navigation bar is a main content area. It starts with a text input field containing 'Enter the Topic : backpropogation'. This is followed by a list of three items, each in a separate box: ':: 1 Intro to BST', ':: 2 tree traversal', and ':: 3 Insert/Delete operation...'. Below these is a button labeled '+ add document for refrence'. At the bottom of the list is a text input field with the placeholder 'How many slides do want?'. A large blue button labeled 'Generate Slide' is positioned at the bottom center of the main content area.

LOW-FIDELITY WIREFRAMES

AFTER FEEDBACK

From TA's Perspective

The wireframe shows a top navigation bar with the label 'AURA' and five buttons: 'create question', 'Queries', 'Slides' (highlighted in orange), 'Mentor', and 'Resources'. Below the navigation bar is a main content area with a text input field containing 'Enter the Topic : backpropogation'. Below this is a list of three items, each preceded by '::': '1 Intro to BST', '2 tree traversal', and '3 Insert/Delete operation...'. Below the list is a button labeled '+ add document for refrence'. At the bottom of the main content area is a text input field containing 'How many slides do want?'. Below this is a blue button labeled 'Generate Slide'.

From Student's Perspective:

The wireframe shows a top navigation bar with the label 'AURA' and four buttons: 'Dashboard', 'Chat Bot', 'Resources' (highlighted in orange), and 'Forum'. Below the navigation bar is a main content area with four items, each in a separate box: 'Lecture video', 'Course Notes', 'Our recommendation', and a bell icon followed by the text 'Take a break: set timer'.

Milestone 3

SCHEDULING AND DESIGN

PROJECT SCHEDULE

TASK DISTRIBUTION

Milestone	Tasks	Start Date	End Date	Assigned To
Milestone 1	Identify Users	01 Oct-25	05 Oct-25	All
	Define User Stories	04 Oct-25	14 Oct-25	All
	Draft and Submit Milestone 1 Report	15 Oct-25	20 Oct-25	Nooha, Aryan, Taniya
Milestone 2	Design Storyboards	16 Oct-25	18 Oct-25	Nooha, Aryan, Taniya
	Design Low-Fidelity Wireframe	15 Oct-25	19 Oct-25	Taniya, Laxmi, Aryan, Vikas
	Draft and Submit Milestone 2 Report	17 Oct-25	20 Oct-25	Vikas, Taniya, Aryan
Milestone 3	Project Scheduling using Gantt and Kanban Board	31 Oct-25	02 Nov-25	Mohit, Aryan
	Component Design	22 Oct-25	27 Oct-25	Mohit, Vikas, Aryan
	Software Design - Class Diagram	30 Oct-25	03 Nov-25	Mohit, Imran, Aryan
	Frontend Development	25 Oct-25	30 Oct-25	Vikas, Aryan, Taniya
	Draft and Submit Milestone 3 Report	01 Nov-25	05 Nov-25	All

PROJECT SCHEDULE

TASK DISTRIBUTION

Milestone	Tasks	Start Date	End Date	Assigned To
Milestone 4	Design APIs	06 Nov-25	09 Nov-25	Laxmi,Mohit, Imran
	Create YAML File	10 Nov-25	15 Nov-25	Aryan, Vikas
	Integrate GenAI Features	08 Nov-25	15 Nov-25	Mohit,Imran, Aziz
	Draft and Submit Milestone 4 Report	13 Nov-25	16 Nov-25	All
Milestone 5	Prepare Test Cases for each API Endpoints	17 Nov-25	20 Nov-25	Aryan, Mohit
	Test API Endpoints	21 Nov-25	23 Nov-25	Vikas, Imran
	Test GenAI Functionality	22 Nov-25	27 Nov-25	Aziz, Mohit
	Draft and Submit milestone 5 Report	25 Nov-25	30 Nov-25	Laxmi, Nooha, Taniya
Milestone 6	Complete Implementation and Final Report	01 Dec-25	04 Dec-25	Mohit, Vikas, Aziz, Imran, Aryan
	Draft and Submit Final Report	04 Dec-25	06 Dec-25	Mohit, Laxmi
	Prepare and Submit Final Presentation	06 Dec-25	07 Dec-25	Nooha, Taniya, Vikas

PROJECT SCHEDULE

SPRINT BACKLOG

SE Team 12

SummaryBacklogBoardCalendarTimelinePagesFormsReports

Search backlog

Filter

SCRUM Sprint - Milestone 1 1 Oct – 20 Oct (6 work items)

000Complete sprint

Understand user needs and document them using structured Agile tools.

SCRUM-16

Conduct user interviews

REQUIREMENT GATH...

DONE

12 Oct

SCRUM-3

Create User Survey Form

REQUIREMENT GATH...

DONE

14 Oct

SCRUM-17

Users identification

REQUIREMENT GATH...

DONE

5 Oct

SCRUM-4

Reach IIC Team for Requirements

REQUIREMENT GATH...

DONE

14 Oct

SCRUM-19

User Stories written using SMART guidelines.

REQUIREMENT GATH...

DONE

16 Oct

SCRUM-5

Finalize the features

REQUIREMENT GATH...

DONE

15 Oct

Create

SCRUM Sprint 2 - Milestone 2 10 Oct – 20 Oct (4 work items)

000Complete sprint

Design intuitive user interfaces and validate with feedback.

SCRUM-6

Story board for the application - ppt or video format

UI/UX

DONE

18 Oct

SCRUM-7

low-fidelity wireframes with each user stories

UI/UX

DONE

18 Oct

SCRUM-9

Feedback from users regarding features and wireframe

UI/UX

DONE

19 Oct

SCRUM-10

Consolidate the wireframe is single pdf

UI/UX

DONE

20 Oct

Create

SCRUM Sprint 3 - Milestone 3 21 Oct – 5 Nov (8 work items)

000Complete sprint

Formalize architecture, scheduling, and frontend development.

SCRUM-11

Design of Components

UI/UX

DONE

30 Nov

SCRUM-13

Software designing - class diagram

UI/UX

DONE

3 Nov

SCRUM-41

Student (AI assistance dashboard UI)

UI/UX

DONE

30 Oct

SCRUM-42

Student(My course and resource dashboard UI)

UI/UX

DONE

30 Oct

SCRUM-43

Student(My Query and dashboard UI)

UI/UX

DONE

30 Oct

SCRUM-15

Gannt chart and kanban board

PROJECT SCHEDULING

DONE

2 Nov

SCRUM-12

Sprint Schedule

PROJECT SCHEDULING

IN PROGRESS

SCRUM-14

Details of MoM

SCRUM

TO DO

SCRUM Sprint 4 - Milestone 4 6 Nov – 16 Nov (12 work items)

000Complete sprint

Implement backend APIs and perform functional testing.

SCRUM-63

Setup Backend

DEVELOPMENT

DONE

6 Nov

SCRUM-65

Add Error Handling and Logging

DEVELOPMENT

DONE

10 Nov

SCRUM-83

Slides generation Agent

DEVELOPMENT

DONE

13 Nov

SCRUM-86

Assessment generator agent

DEVELOPMENT

DONE

11 Nov

SCRUM-70

Compile Milestone 4 Document

DEVELOPMENT

DONE

14 Nov

SCRUM-64

Design API Endpoints

DEVELOPMENT

DONE

6 Nov

SCRUM-82

RAG Agent Creation

DEVELOPMENT

IN PROGRESS

12 Nov

SCRUM-66

Write Unit Tests

DEVELOPMENT

IN PROGRESS

9 Nov

SCRUM-67

Document Test Results

DEVELOPMENT

IN PROGRESS

11 Nov

SCRUM-68

Create Swagger-Compatible YAML File

DEVELOPMENT

DONE

12 Nov

SCRUM-69

Prepare Test Case Document (PDF)

DEVELOPMENT

IN PROGRESS

13 Nov

SCRUM-87

Input field for different roles

DEVELOPMENT

DONE

11 Nov

Create

PROJECT SCHEDULE

SPRINT BACKLOG

☐ SCRUM Sprint 5 - Milestone 5 17 Nov – 30 Nov (2 work items) 000 Start sprint ... Integrate all milestones, finalize documentation, and present project.									
☒	SCRUM-72	Capture Test Execution Screenshots	TESTING	TO DO	20 Nov	-	=		
☒	SCRUM-85	php	UI/UX	TO DO		-	=		
+ Create									
2 of 2 work items visible Estimate: 0 of 0									

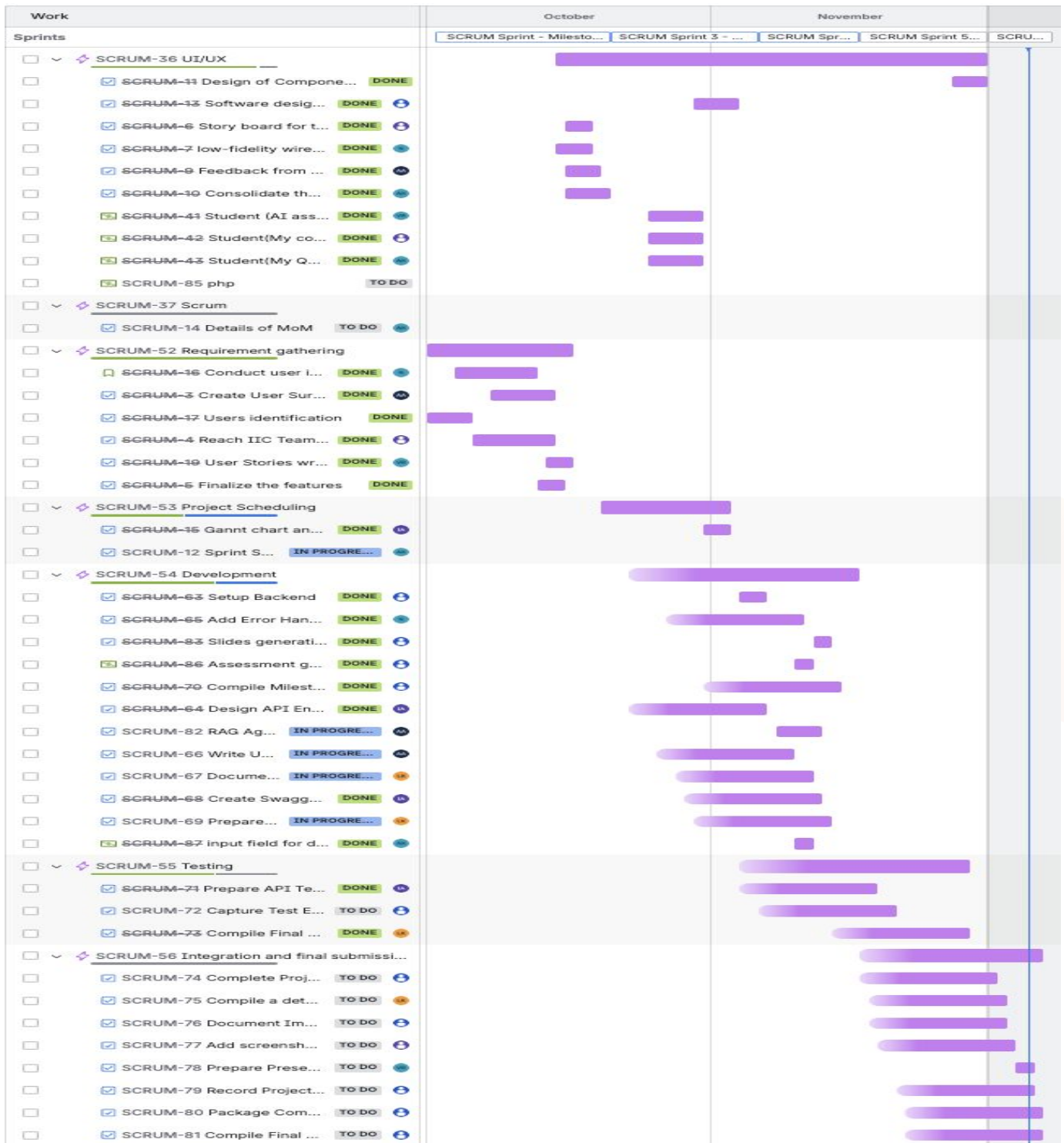
☐ SCRUM Sprint 6 - Milestone 6 30 Nov – 7 Dec (8 work items) 000 Start sprint ... Final submission									
☒	SCRUM-74	Complete Project Implementation & Prototype	INTEGRATION AND F...	TO DO	1 Dec	-	=		
☒	SCRUM-75	Compile a detailed report including all milestones, implementation details, tools used, and run instruction	INTEGRATION AND F...	TO DO	2 Dec	-	=		
☒	SCRUM-76	Document Implementation Details	INTEGRATION AND F...	TO DO	2 Dec	-	=		
☒	SCRUM-77	Add screenshots and descriptions of code review process, issue reporting, and tracking mechanisms	INTEGRATION AND F...	TO DO	3 Dec	-	=		
☒	SCRUM-78	Prepare Presentation Slides	INTEGRATION AND F...	TO DO	4 Dec	-	=		
☒	SCRUM-79	Record Project Demonstration Video	INTEGRATION AND F...	TO DO	5 Dec	-	=		
☒	SCRUM-80	Package Complete Codebase	INTEGRATION AND F...	TO DO	6 Dec	-	=		
☒	SCRUM-81	Compile Final PDF Report	INTEGRATION AND F...	TO DO	6 Dec	-	=		

☐ Backlog (10 work items) 000 Create sprint ...									
☒	SCRUM-57	Design API		TO DO		-	=		
☒	SCRUM-46	Create Project Schedule		IN PROGRESS		-	=		
☒	SCRUM-47	Build UML Class Diagram		IN PROGRESS		-	=		
☒	SCRUM-48	Prepare Milestone 2 Report		IN PROGRESS		-	=		
☒	SCRUM-49	Create Gantt chart and Kanban Board		IN PROGRESS		-	=		
☒	SCRUM-58	prepare test cases for each API endpoint		TO DO		-	=		
☒	SCRUM-59	Test API endpoints		TO DO		-	=		
☒	SCRUM-60	Backend Setup		IN PROGRESS		-	=		
☒	SCRUM-61	Deployment of application		TO DO		-	=		
☒	SCRUM-62	Milestone 3 report creation		IN PROGRESS		-	=		
+ Create									

PROJECT SCHEDULE

GANTT CHART

A Gantt Chart was created in Jira to represent the project's timeline and progress, divided into multiple sprints. This helped in visually organizing tasks, their durations, dependencies, and completion status, ensuring that all team members are aligned with the milestones and deadlines.



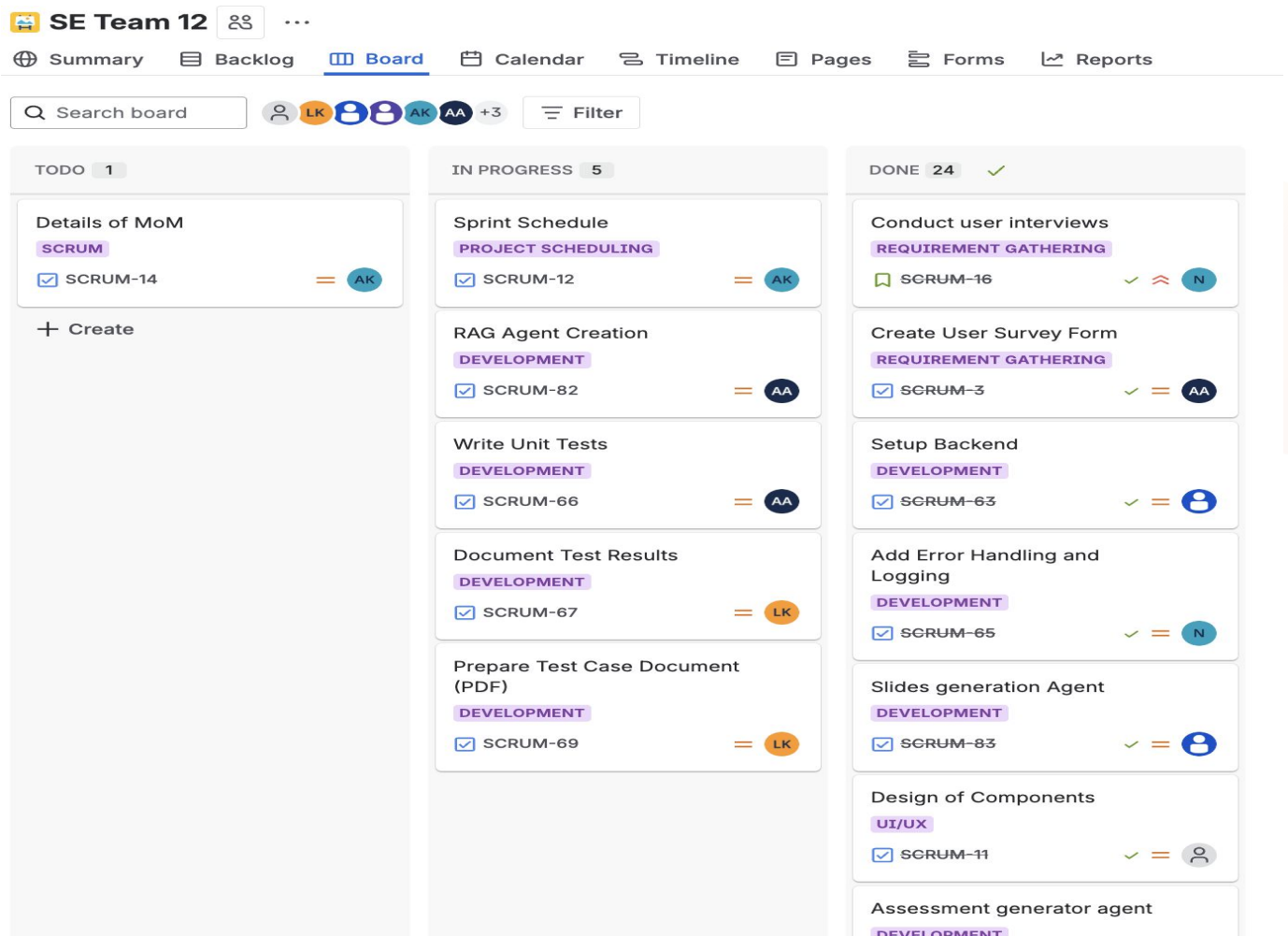
PROJECT SCHEDULE

KANBAN BOARD

The team utilized a Kanban board in Jira to efficiently manage sprint tasks, monitor progress, and ensure clear accountability throughout Milestone 3. Each task was categorized under columns such as To Do, In Progress, Review, and Done, reflecting the real-time development status across all modules – Student, TA, Instructor, and Admin.

The board helped maintain continuous visibility of the project workflow, enabling the team to prioritize issues, resolve blockers promptly, and achieve balanced workload distribution.

By the end of this milestone, the Kanban board reflected substantial progress, with key tasks for UI implementation, component design, and documentation successfully transitioned to the Done phase, ensuring timely milestone completion.



SCRUM MEETINGS

SCRUM 1 - SCRUM 3

Team Formation & Orientation (17 Sep – 23 Sep)

Focus: Team introduction, role assignment, and domain exploration.

Summary: Team members introduced themselves and discussed prior experience with Vue.js, Python, and database technologies. Reviewed previous year project reports and problem statements for inspiration.

Discussed project domains and finalized Customer Service as the broad sector for exploration.

Assigned preliminary roles based on technical strengths (Frontend, Backend, Documentation, UI/UX).

Action Items: Each member to research ideas from different sectors (education, healthcare, e-commerce, academic services).

Prepare short presentations for the next scrum to pitch potential project ideas.

SCRUM 4- SCRUM 6

Problem Statement & Idea Finalization (02 Oct – 10 Oct)

Focus: Idea evaluation, user identification, and initial user story collection.

Summary: The team explored all presented ideas and narrowed focus to the Academic Service domain under customer service.

Defined the problem statement around “AI-powered academic service automation.”

Conducted user interviews with students, TAs, and instructors to collect initial user stories.

Set up Jira Kanban Board for task tracking and discussed agile workflow.

Drafted initial Milestone 1 submission report and presented key ideas to the client.

The client emphasized focusing on provider-side challenges and improving user story clarity.

Action Items: Finalize SMART-based problem statement.

Conduct additional interviews focusing on secondary users (TAs, instructors, and admins).

Refine user stories and categorize by user type (Primary, Secondary, Tertiary).

SCRUM MEETINGS

SCRUM 7 - SCRUM 9

Feature Finalization & Storyboarding (10 Oct – 20 Oct)

Focus: Refining user stories, defining features, and preparing for wireframe creation.

Summary: Collected more user stories from secondary users (TAs and Instructors). Finalized ten AI-based features based on story alignment and feasibility. Client suggested adding two additional features (Query Escalation System and Analytics Dashboard). Began drafting wireframes and storyboards for selected features based on finalized user stories. Started organizing user stories under relevant features and GenAI roles.

Action Items: Submit the updated feature list and user story document for client review.

Begin visual design for storyboard representation.

Plan project milestones and timelines for Milestone 2.

SCRUM 10 - SCRUM 12

UI/UX Implementation and Component Design (21 Oct – 04 Nov)

Focus: Project planning, frontend implementation, and component design for Milestone 3 deliverables.

Summary: The team created a comprehensive Project Plan outlining sprint structure, milestone deliverables, and timelines to maintain clarity and accountability across roles. Defined a consistent UI theme and design style, following IITM's institutional color palette and academic branding standards. Developed and presented the Class Diagram as part of Milestone 3, ensuring system structure alignment with finalized features. Implemented frontend modules for Student, TA, and Instructor dashboards – achieving approximately 70% completion with routing, left-side navigation, and responsive design. Began component diagram development and frontend integration work, using dummy data for interface testing. Prepared internal Scrum Minutes and GitHub issue logs to maintain transparent communication and task tracking.

Action Items: Submit Project Plan to the client for feedback and milestone verification. Review and refine the Class Diagram by 31 October 2025 to incorporate client feedback. Design and finalize 7 key component diagrams (Dashboard, Chat Interface, Query Tracker, Analytics Panel, Slide Generator, Content Recommender, and Onboarding Assistant). Conduct internal UI testing and review, ensuring consistent navigation, responsiveness, and error-state handling. Prepare and submit Milestone 3 Report by 4 November 2025, including screenshots, diagrams, and the frontend code package (with README instructions).

SOFTWARE DESIGN - SYSTEM COMPONENTS

BACKEND ARCHITECTURE & API ENDPOINTS

Base URL Convention

All routes prefixed as:

/api/v1/

Authentication & User Management

Method Endpoint Description

POST /auth/register Register new user (student/TA/instructor/admin)

POST /auth/login Login and return JWT access token

POST /auth/logout Invalidate token / logout

GET /auth/me Get current user profile

PATCH /auth/update Update user profile details

POST /auth/refresh Refresh access token

AI Chat & Query Handling (RAG Assistant)

Method Endpoint Description

POST /chat/ask Ask academic/admin question (RAG-powered chatbot)

GET /chat/history Retrieve user's previous chat sessions

POST /chat/feedback Submit feedback for AI response

GET /chat/context Retrieve context docs or references used in last answer

Smart Study Assistant(Summaries,Deadlines, Reminders)

Method Endpoint Description

GET /study/summary/weekly

Generate or fetch weekly summary of lectures, quizzes, announcements

POST /study/reminders/create Create smart study-break reminder

PATCH /study/reminders/update Update smart study-break reminder

DELETE /study/reminders/delete Delete smart study-break reminder

GET /study/reminders List all active reminders

POST /study/deadlines/add Add project, quiz, or admin deadline

GET /study/deadlines Get user's academic calendar

SOFTWARE DESIGN - SYSTEM COMPONENTS

BACKEND ARCHITECTURE & API ENDPOINTS

AI Content Recommender

Method Endpoint Description

GET /recommend/content

Fetch AI-based learning resource recommendations

GET /recommend/related Suggest external resources for given topic

POST /recommend/feedback Log user engagement or usefulness feedback

TA Doubt Summarizer

Method Endpoint Description

POST /ta/doubts/upload Upload student chat/forum/email data

GET /ta/doubts/summary Get summarized and clustered student doubts

GET /ta/doubts/topics View most confusing topics (cluster analysis)

GET /ta/doubts/insights Get LLM-driven insights on learning gaps

AI Query Escalation & Tracking

Method Endpoint Description

POST /queries/submit Submit new academic/admin/technical query

GET /queries View user queries and status

GET /queries/unresolved For TA/Admin: fetch unresolved queries

PATCH /queries/assign/:id Assign query to TA/Instructor

PATCH /queries/status/:id Update query status (pending/resolved/escalated)

Admin Analytics Dashboard

Method Endpoint Description

GET /analytics/overview Get overall metrics (queries, responses, users)

GET /analytics/faqs Top FAQs or recurring questions

GET /analytics/performance Response time and resolution rate

GET /analytics/sentiment Aggregate feedback sentiment

GET /analytics/usage Active users, API calls, session stats

AI Discourse Integration (Forums & Threads)

Method Endpoint Description

GET /discourse/threads List forum discussions by topic

GET /discourse/summary/:topic Summarize discussion threads on a topic

GET /discourse/search Semantic search through forum history

POST /discourse/link Link related discussions together

SOFTWARE DESIGN - SYSTEM COMPONENTS

BACKEND ARCHITECTURE & API ENDPOINTS

TA Onboarding Mentor

Method Endpoint Description

POST /mentor/chat Ask onboarding or teaching-related questions

GET /mentor/resources Get curated onboarding materials

GET /mentor/tips Fetch daily or contextual teaching tips

Automated Assessment Generator

Method Endpoint Description

POST /assessment/generate Generate question paper by topic/difficulty

GET /assessment/templates Fetch available question templates

PATCH /assessment/edit/:id Modify generated questions manually

POST /assessment/export Export final question paper (PDF/DOCX)

Slide Deck Generator

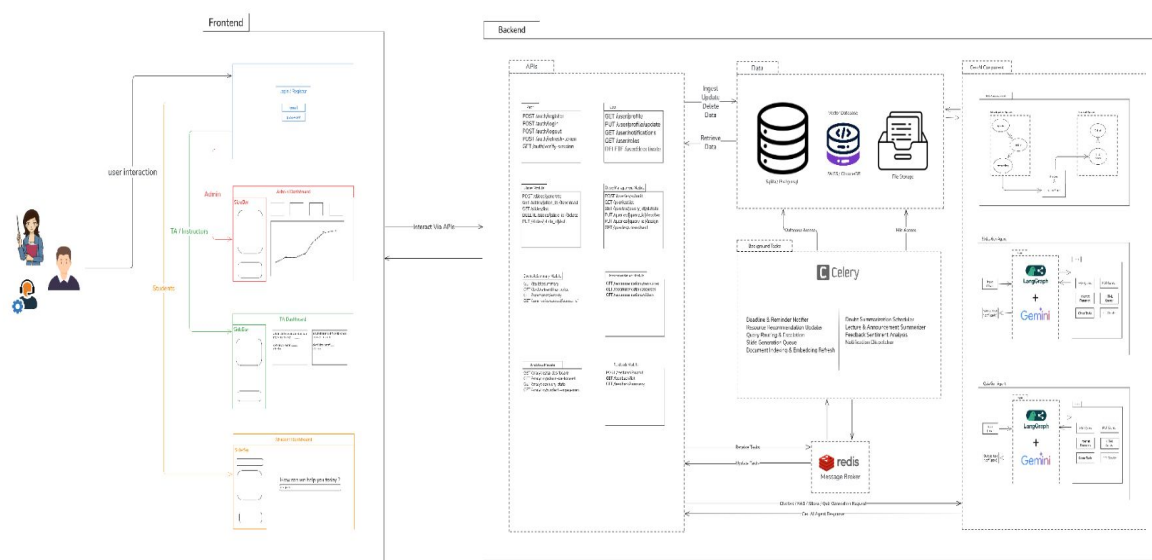
Method Endpoint Description

POST /slides/generate Generate teaching slides from structured content

GET /slides/history View generated slide deck history

POST /slides/upload Upload custom templates

GET /slides/download/:id Download generated slide deck



SOFTWARE DESIGN - SYSTEM COMPONENTS

FRONTEND

TA/Instructor View (Query Response & Content Creation)

View assigned queries

- Dashboard of queries assigned to them
- Filter by priority, category, and status
- View student query history

Respond to queries

- Start chat sessions for query resolution
- Update query status (In Progress, Resolved)
- Add detailed responses with file attachments

Generate educational content

- Use AI to create slide decks from topics
- Generate assessments with customizable questions
- Upload and manage learning resources

Monitor student engagement

- View query response times
- Track student participation in chats
- Access content usage analytics

Manage notifications

- Set preferences for query alerts
- Receive updates on new queries and deadlines

SOFTWARE DESIGN - SYSTEM COMPONENTS

FRONTEND

Student View (Query Management & Learning)

Create a new query

- Select category (academic, technical, general)
- Set priority level
- Add message description with file attachments

View and manage queries

- Dashboard to view all submitted queries
- Filter by status (Assigned, In Progress, Resolved)
- Track query assignment to TA/Instructor

Chat with TA/Instructor

- Real-time messaging for query resolution
- View chat history and context documents
- Receive notifications for responses

Access learning resources

- Browse slide decks, assessments, and materials
- Download content for offline access
- Rate and provide feedback on resources

Manage reminders

- Create reminders for deadlines and exams
- Receive notifications for query updates
- View reminder dashboard

SOFTWARE DESIGN - SYSTEM COMPONENTS

FRONTEND

Admin/Developer View (System Management)

User management

- Create and manage user accounts
- Assign roles (Student, TA, Instructor, Admin)
- View user activity logs

System monitoring

- View platform usage statistics
- Monitor query resolution metrics
- Track AI content generation usage

Content moderation

- Review and approve user-generated content
- Manage content categories and tags
- Handle reported issues

System configuration

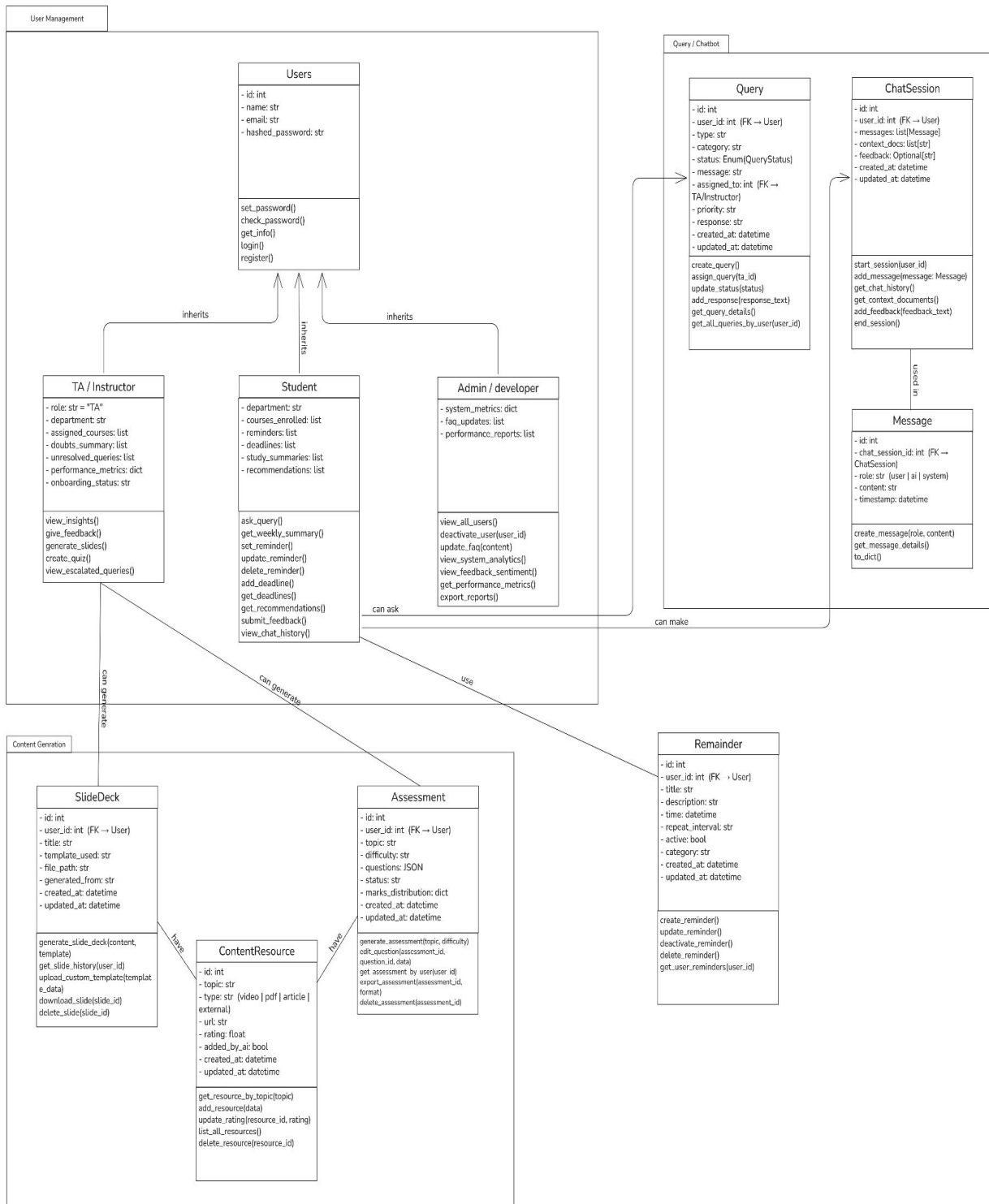
- Update notification templates
- Configure AI service parameters
- Manage database backups and storage

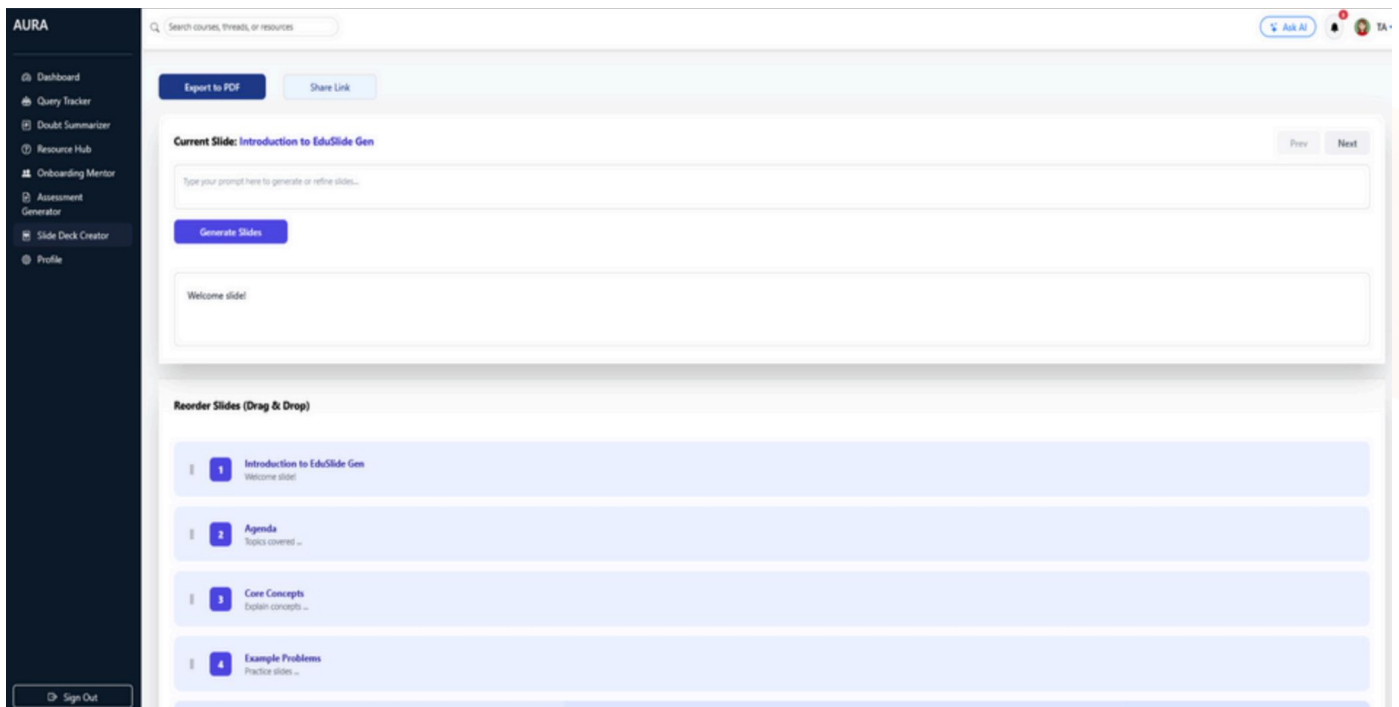
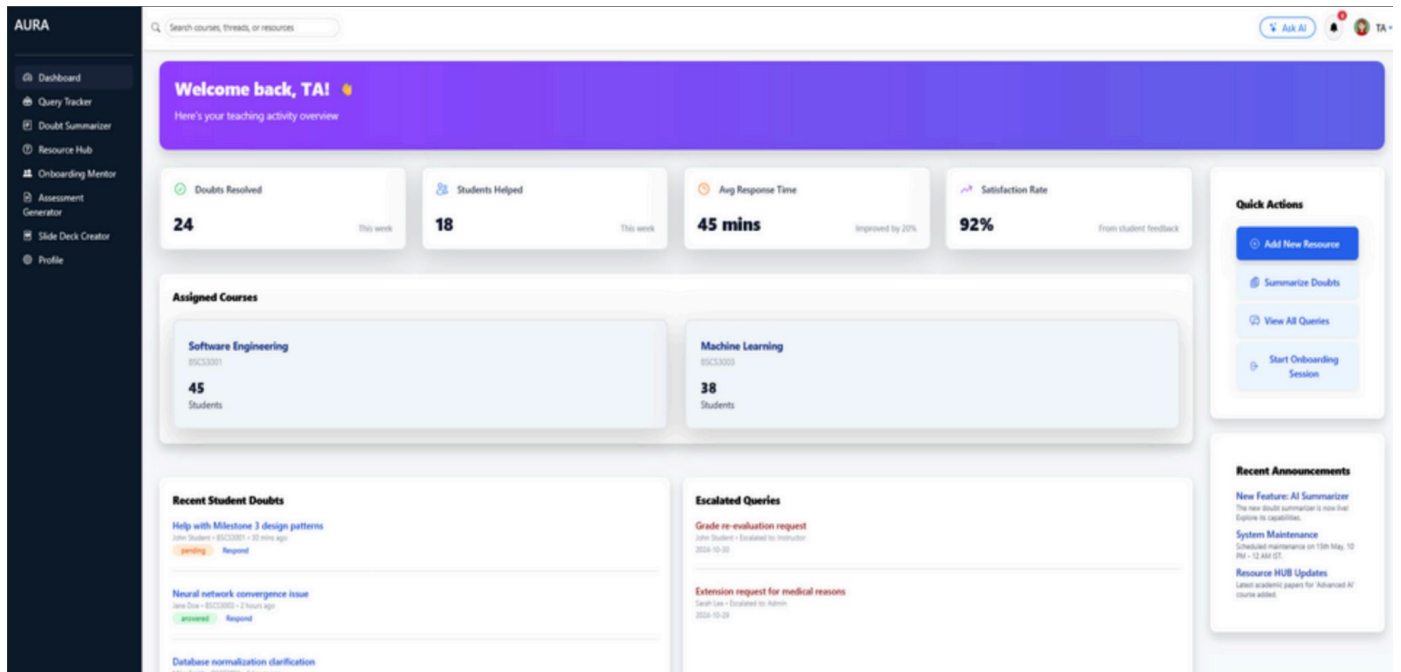
Analytics and reporting

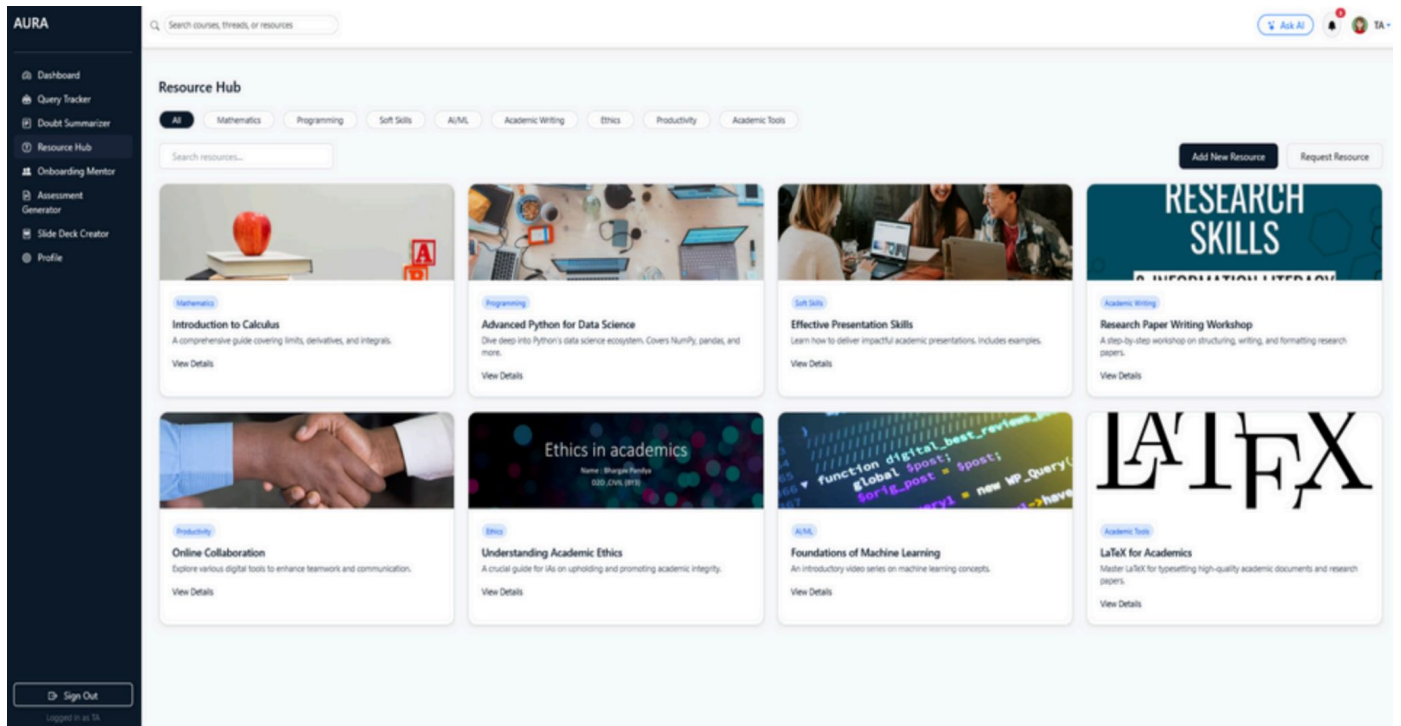
- Generate system-wide performance reports
- Export data in PDF/CSV formats
- View user engagement trends

CLASS DIAGRAM

Class Diagram

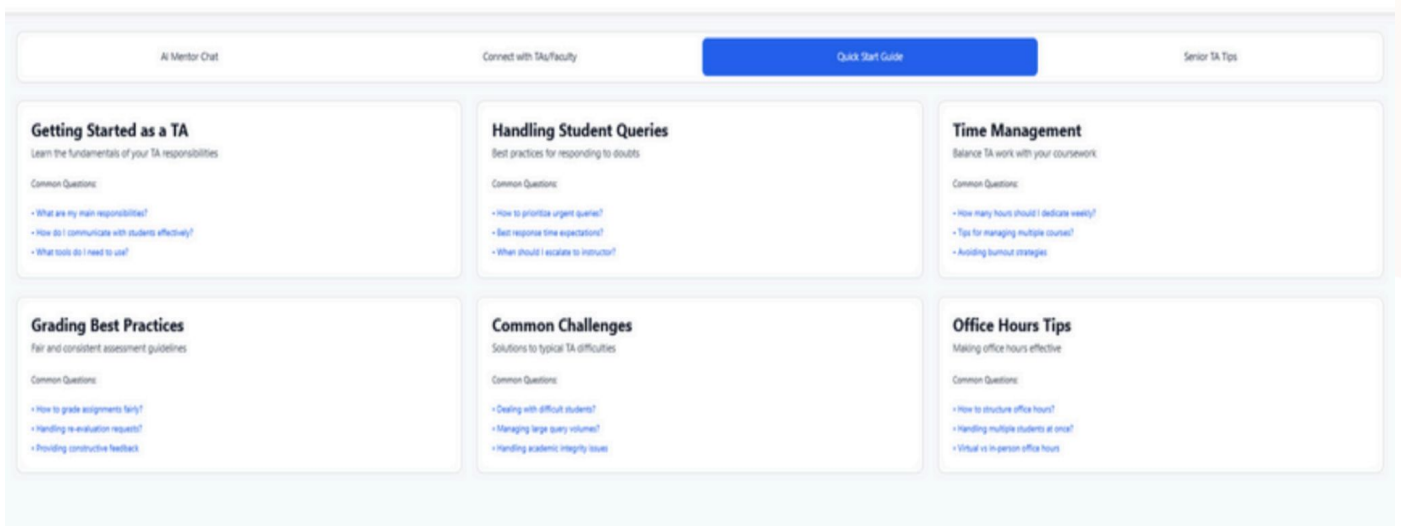


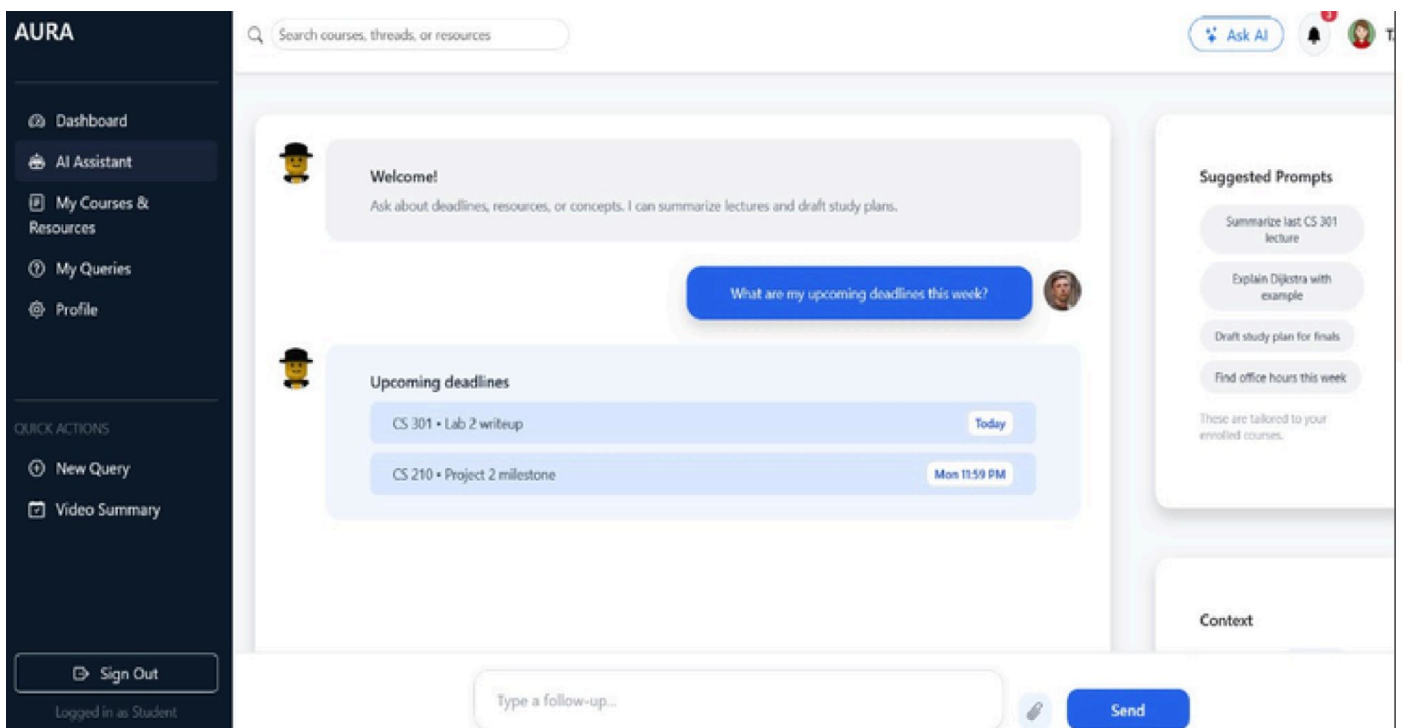
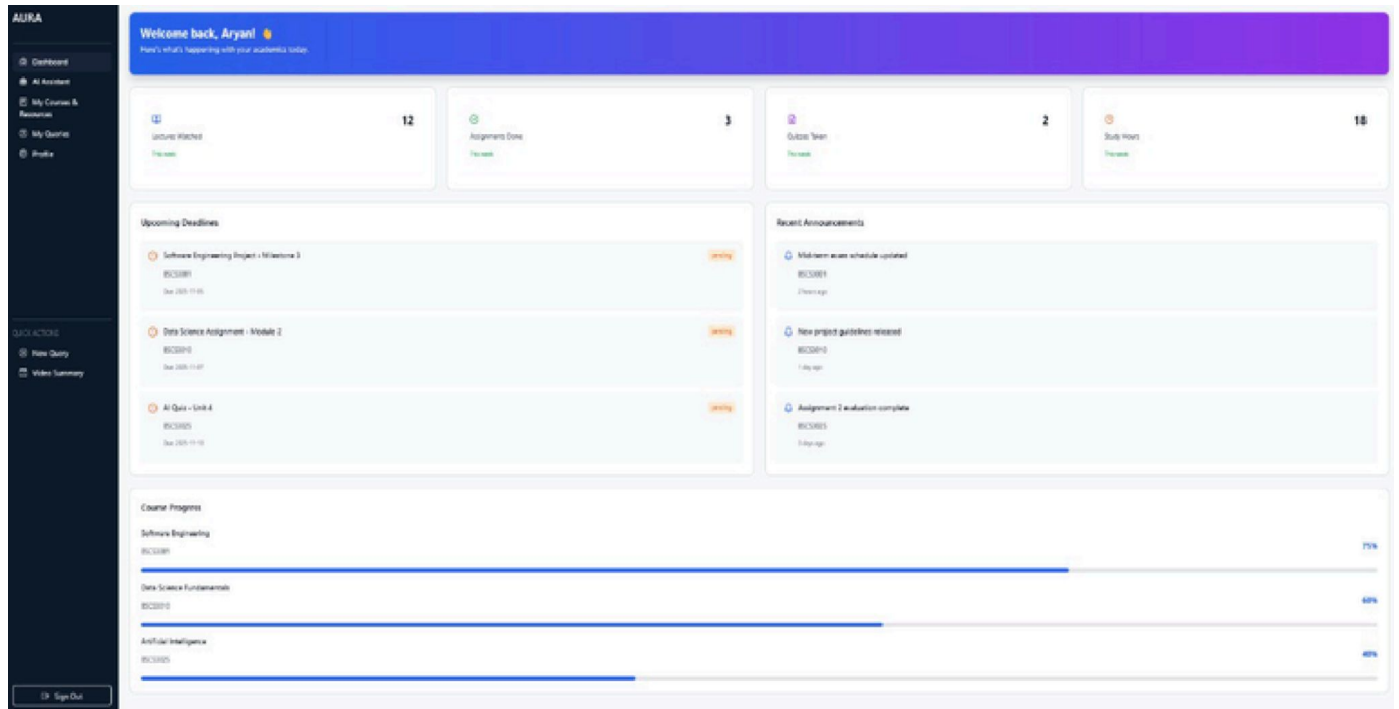


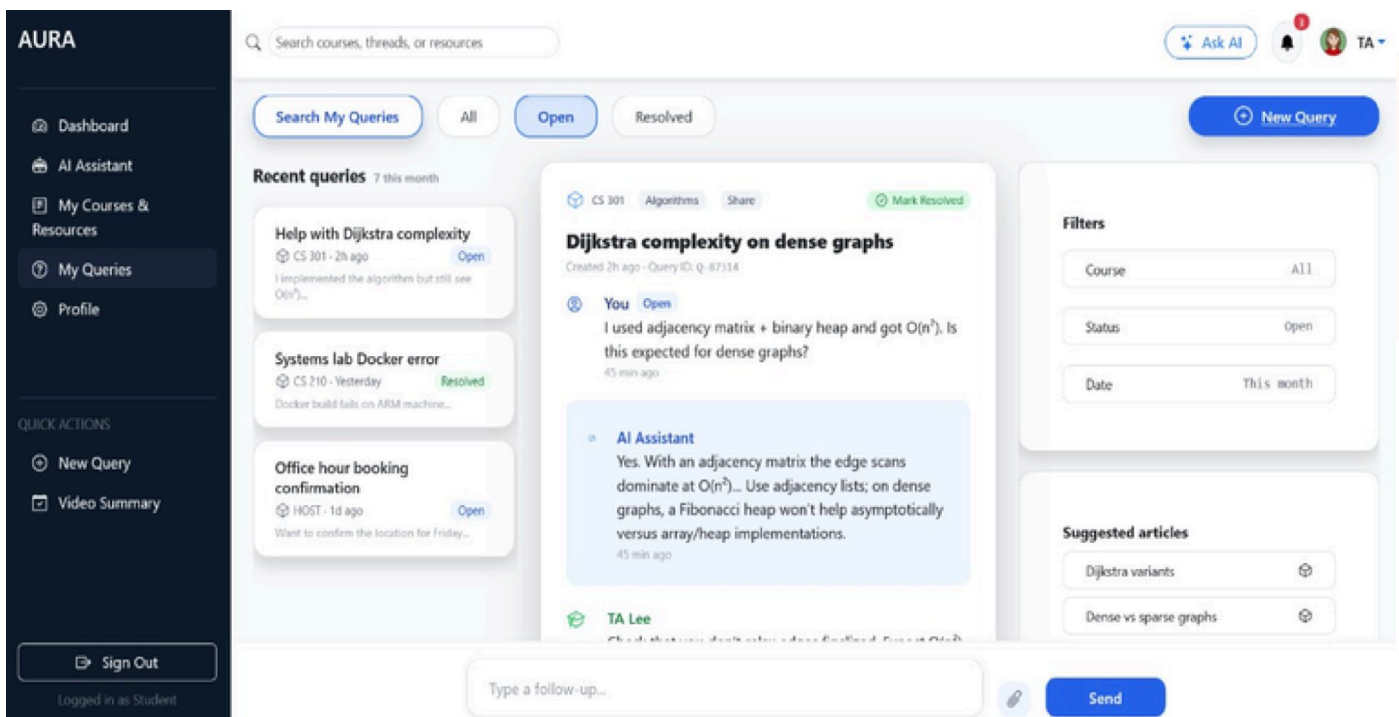
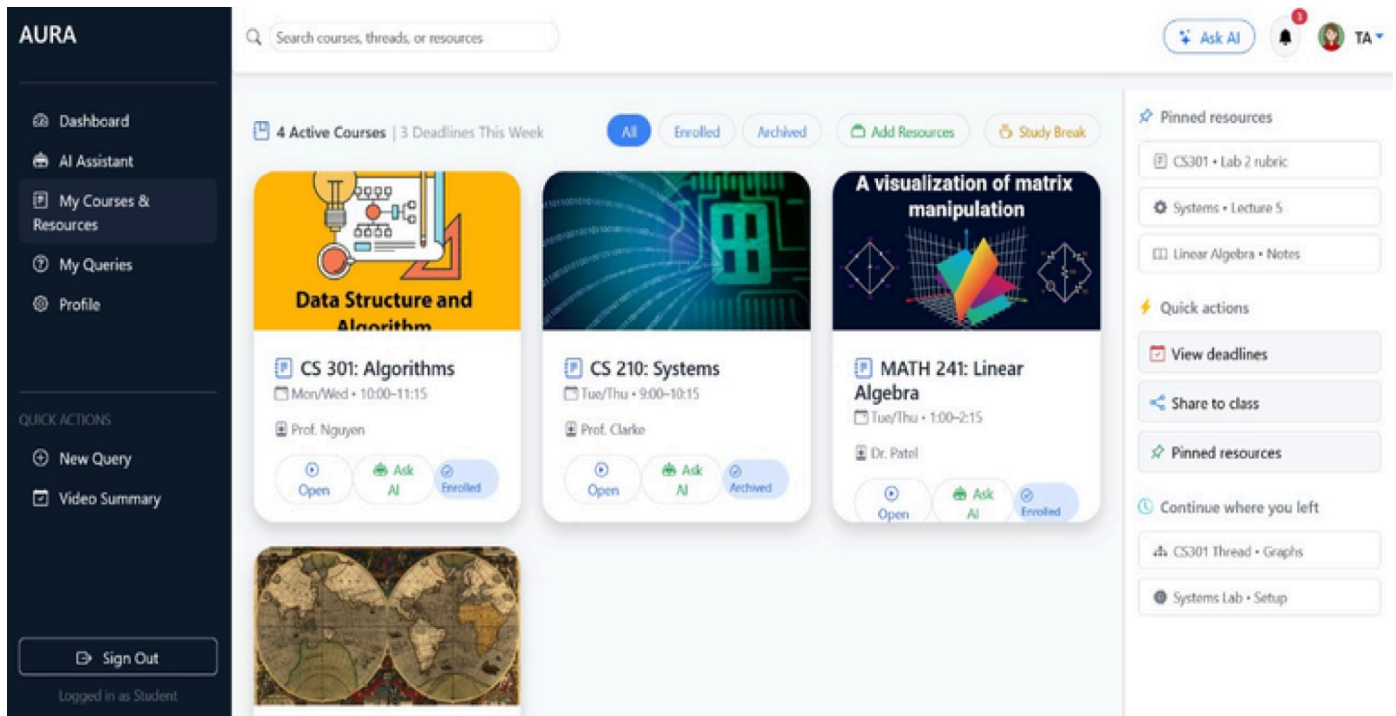


AI Onboarding Mentor

Your personalized guide trained on institutional best practices, senior TA experiences, and faculty wisdom







AURA

Dashboard

AI Assistant

My Courses & Resources

My Queries

Profile

QUICK ACTIONS

New Query

Video Summary

Sign Out

Logged in as Student

Search courses, threads, or resources

Ask AI

TA

Video Summary Generator

Enter Video Link (YouTube, Vimeo, etc.)

https://www.youtube.com/watch?v=...

Generate

Summary

Enter a video link above to get the summary.

Welcome back, Admin! 🎉

Here's a birds-eye view of your institution for today.

1,276

Active Users

86

Open Queries

19

Flagged Issues

99%

System Health

FAQ Analytics

FAQ Frequency

Admissions

Fees

Exams

Top-asked questions and answer accuracy.

Unresolved Queries

Academic

Technical

Exam

Monitor backlog and escalation rate.

Quick Actions

Send Announcement

Export Reports

Manage Roles

Recent System Announcements

Resource HUB update added for 'Advanced AI'

New feature: AI Query Summarizer is now live!

Scheduled Maintenance: Nov 10, 12 AM – 2 AM

Recent Institutional Queries

Export

ID	Topic	Role	Date	Status
3012	Exam Dates Change	Student	2025-11-03	Pending
1989	Resource Access Issue	Instructor	2025-11-02	Resolved
2244	Duplicate Queries	TA	2025-11-01	Escalated
2967	Policy Suggestion	Student	2025-11-01	Pending

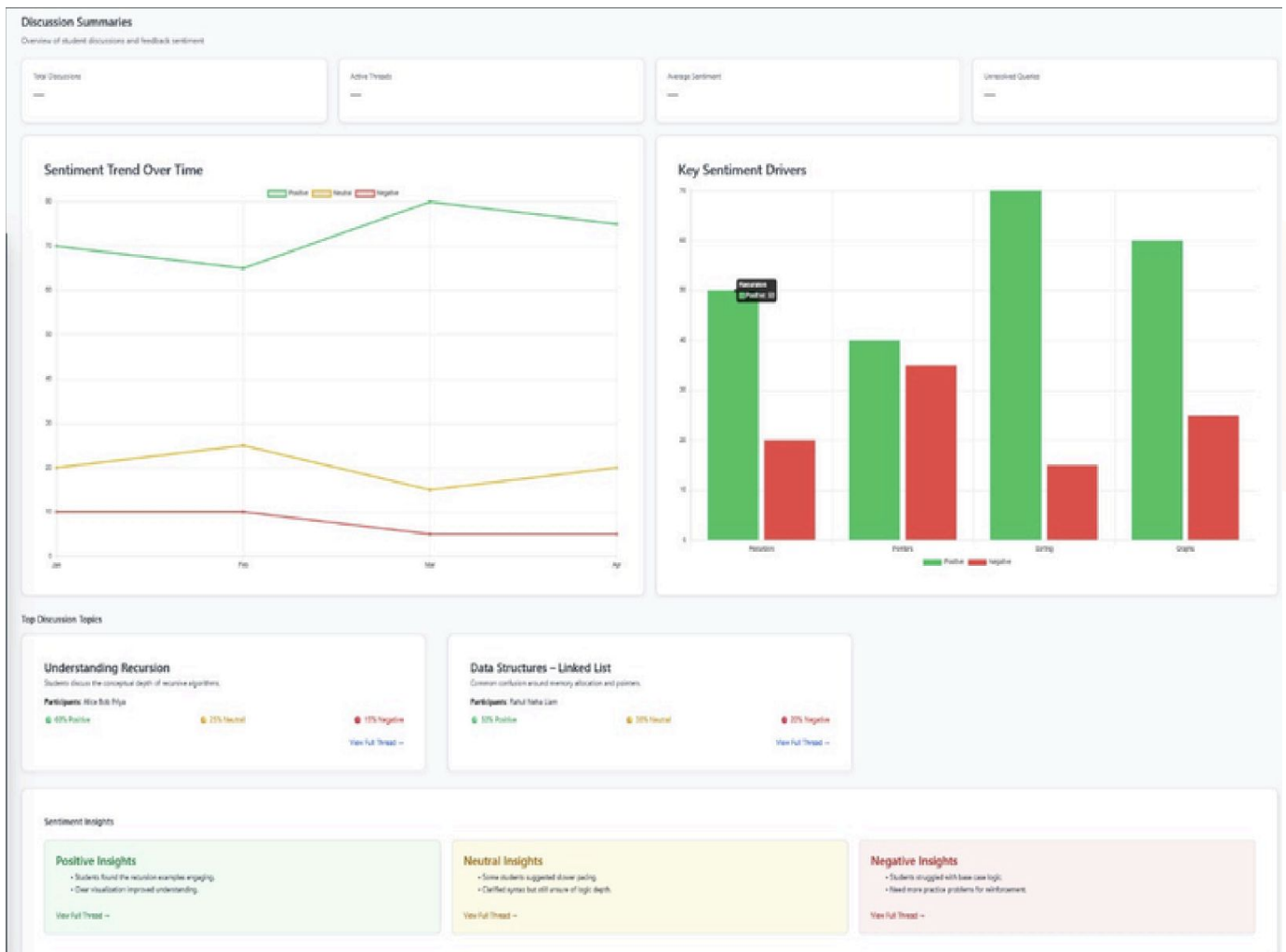
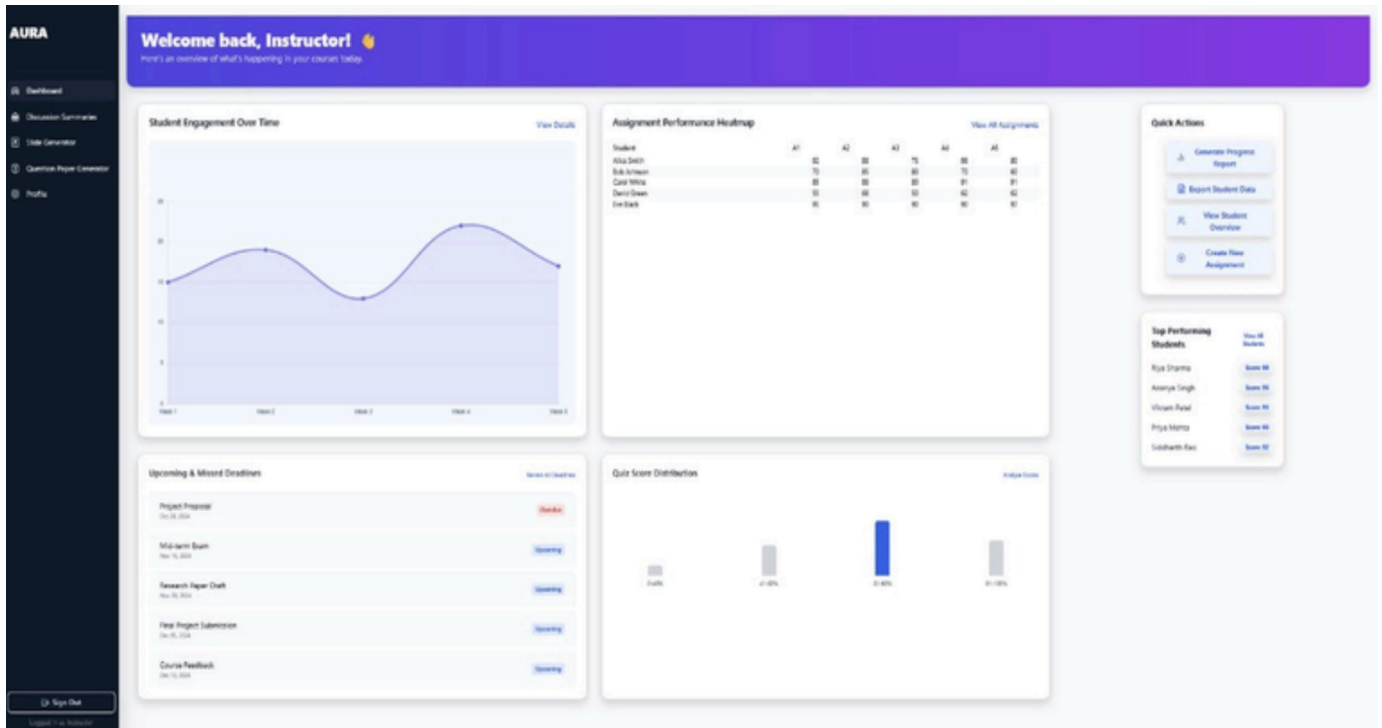
User Management

Bulk Approve

User	Email	Role	Status	Actions
Jane Doe	jane@iitm.ac.in	TA	Pending	Approve Block Flag
Dr. S. Kalra	kalra@iitm.ac.in	Instructor	Active	Block Flag
Ankur Sharma	ankur@iitm.ac.in	Student	Blocked	Flag
Priya Mehta	priya@iitm.ac.in	Student	Active	Block Flag

Milestone 6
TEAM 12

46



Milestone 4

API ENDPOINTS

API ENDPOINTS

Authentication



POST	/api/auth/signup	Register a new user	✓
POST	/api/auth/login	Login and obtain tokens	✓
POST	/api/auth/refresh	Refresh access token	✓
GET	/api/auth/me	Get current user profile	🔒 ✓
PUT	/api/auth/me	Update current user profile	🔒 ✓

Doubt Summarizer



POST	/api/ta/doubts/upload	Upload Doubts	✓
GET	/api/ta/doubts/summary/{course_code}	Get Doubt Summary	✓
GET	/api/ta/doubts/topics/{course_code}	Get Topic Clusters	✓
GET	/api/ta/doubts/insights/{course_code}	Get Teacher Insights	✓
GET	/api/ta/doubts/source-breakdown/{course_code}	Get Source Breakdown	✓
GET	/api/ta/doubts/export/pdf	Export Summary Pdf	✓
GET	/api/ta/doubts/export/csv	Export Summary Csv	✓
POST	/api/ta/doubts/export/email	Email Summary Report	✓

API ENDPOINTS

Chatbot



POST	/api/chatbot/chat	Chat with AI assistant		
POST	/api/chatbot/chat/stream	Stream chat response		
DELETE	/api/chatbot/conversation/{conversation_id}	Clear conversation history		
GET	/api/chatbot/conversation/{conversation_id}/history	Get conversation history		
GET	/api/chatbot/status	Get chatbot status		
GET	/api/chatbot/metrics	Get chatbot metrics		
GET	/api/chatbot/conversation/{conversation_id}/state	Get conversation session state		
POST	/api/chatbot/chat/enhanced	Enhanced chat with knowledge base integration		
POST	/api/chatbot/chat/enhanced/stream	Enhanced streaming chat with knowledge base		
GET	/api/chatbot/search-knowledge	Search knowledge base		
POST	/api/chatbot/answer-query/{query_id}	Answer a specific query using AI and knowledge base		
GET	/api/chatbot/user-context	Get user context for chatbot personalization		

API ENDPOINTS

Knowledge Base



POST	/api/knowledge/sources	Create Knowledge Source		
GET	/api/knowledge/sources	List Knowledge Sources		
GET	/api/knowledge/categories	Get Categories		
GET	/api/knowledge/sources/{source_id}	Get Knowledge Source		
PUT	/api/knowledge/sources/{source_id}	Update Knowledge Source		
DELETE	/api/knowledge/sources/{source_id}	Delete Knowledge Source		
GET	/api/knowledge/sources/{source_id}/chunks	Get Source Chunks		
POST	/api/knowledge/search	Semantic Search		
GET	/api/knowledge/stats	Get Knowledge Stats		

Dashboard & Analytics



GET	/api/dashboard/statistics	Get Dashboard Statistics		
GET	/api/dashboard/activity-timeline	Get Activity Timeline		
GET	/api/dashboard/top-sources	Get Top Sources		

API ENDPOINTS

Background Tasks



GET	/api/tasks/	List Tasks		
GET	/api/tasks/{task_id}	Get Task		
DELETE	/api/tasks/{task_id}	Delete Task		
GET	/api/tasks/statistics/summary	Get Task Statistics		

Seed Data



POST	/api/seed/populate	Populate Database		
DELETE	/api/seed/clear	Clear Seed Data		
POST	/api/seed/populate-enhanced	Populate Enhanced Database		
DELETE	/api/seed/clear-all	Clear All Data		

Queries



GET	/api/queries/	List queries		
POST	/api/queries/	Create new query		
GET	/api/queries/{query_id}	Get query details		
POST	/api/queries/{query_id}/response	Add response to query		
PUT	/api/queries/{query_id}/status	Update query status		
GET	/api/queries/statistics/summary	Get query statistics		

API ENDPOINTS

Analytics



GET	/api/analytics/overview	Get overall system metrics		
GET	/api/analytics/faqs	Get frequently asked questions		
GET	/api/analytics/performance	Get performance metrics		
GET	/api/analytics/sentiment	Get sentiment analysis		
GET	/api/analytics/usage	Get usage statistics		

Courses



GET	/api/courses/	List all available courses		
POST	/api/courses/	Create a new course (Admin only)		
GET	/api/courses/public	List all available courses (Public)		
GET	/api/courses/my-courses	Get current user's assigned courses		
GET	/api/courses/{course_id}	Get a course by ID		
PUT	/api/courses/{course_id}	Update a course (Admin only)		
DELETE	/api/courses/{course_id}	Delete a course (Admin only)		

API ENDPOINTS

Tags



POST	/api/tags/	Create a new tag (TA/Admin only)		
GET	/api/tags/	List all tags or search by name		
GET	/api/tags/{tag_id}	Get a tag by ID		
PUT	/api/tags/{tag_id}	Update a tag (TA/Admin only)		
DELETE	/api/tags/{tag_id}	Delete a tag (TA/Admin only)		

Quizzes



POST	/api/quizzes/generate	Generate a new quiz (TA/Admin only)		
PUT	/api/quizzes/{quiz_id}	Update an existing quiz (Creator only)		
GET	/api/quizzes/{quiz_id}	Get a single quiz		
DELETE	/api/quizzes/{quiz_id}	Delete a quiz (Creator only)		
GET	/api/quizzes/	List all quizzes		
POST	/api/quizzes/{quiz_id}/attempt	Submit an attempt for a quiz		
POST	/api/quizzes/{quiz_id}/publish	Publish a quiz (Creator only, manual review mode)		
POST	/api/quizzes/{quiz_id}/unpublish	Unpublish a quiz (Creator only)		

API ENDPOINTS

Slide Decks

POST	/api/slide-decks/preview	Generate a preview of the slide deck (TA/Instructor only)	🔒	✓
POST	/api/slide-decks/	Generate a new slide deck (TA/Instructor only)	🔒	✓
GET	/api/slide-decks/	List all slide decks	🔒	✓
GET	/api/slide-decks/{deck_id}	Get a single slide deck	🔒	✓
PUT	/api/slide-decks/{deck_id}	Update a slide deck (Creator only)	🔒	✓
DELETE	/api/slide-decks/{deck_id}	Delete a slide deck (Creator only)	🔒	✓
GET	/api/slide-decks/{deck_id}/export	Export a slide deck to PPTX or PDF (Creator only)	🔒	✓

Video

POST	/api/video/summarize	Summarize Video Endpoint		✓
------	----------------------	--------------------------	--	---

Student Resources

GET	/api/student-resources/my-resources	Get My Resources	🔒	✓
POST	/api/student-resources/upload-document	Upload Document	🔒	✓
POST	/api/student-resources/upload-image	Upload Image	🔒	✓
POST	/api/student-resources/add-link	Add Link	🔒	✓
DELETE	/api/student-resources/{resource_id}	Delete Resource	🔒	✓
PATCH	/api/student-resources/{resource_id}/pin	Toggle Pin Resource	🔒	✓
GET	/api/student-resources/download/{resource_id}	Download Resource	🔒	✓

API ENDPOINTS

Instructor Analytics



GET	/api/instructor/discussion-summaries	Get discussion summaries for instructor dashboard		
GET	/api/instructor/topic-clusters	Get topic clustering analysis		
GET	/api/instructor/realtime-sentiment	Get real-time sentiment data		
GET	/api/instructor/discussion/{query_id}	Get detailed discussion thread		

Health



GET	/health	Health check endpoint		
-----	---------	-----------------------	--	--

Info



GET	/api	API information		
-----	------	-----------------	--	--

Milestone 5

TEST SUITE

APIs Tests

/api/auth/signup

- **Method:** POST
- **Test Case:** Register a new user account with email and password.

```
def test_signup_success(self, client: TestClient, db_session: Session):
    """Test successful user registration."""
    response = client.post(
        "/api/auth/signup",
        json={
            "email": "newuser@example.com",
            "password": "SecurePass123!",
            "full_name": "New User",
            "role": "student"
        }
    )

    assert response.status_code == 201
    data = response.json()

    # Check response structure
    assert "access_token" in data
    assert "refresh_token" in data
    assert "token_type" in data
    assert data["token_type"] == "bearer"
    assert "expires_in" in data
    assert "user" in data

    # Check user data
    user = data["user"]
    assert user["email"] == "newuser@example.com"
    assert user["full_name"] == "New User"
    assert user["role"] == "student"
    assert user["is_active"] is True
    assert "id" in user
    assert "created_at" in user
```

APIs Tests

Input

```
{
  "email": "user@example.com",
  "password": "stringst",
  "role": "student",
  "full_name": "string"
}
```

Expected Output

```
{
  "access_token":
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "refresh_token":
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "token_type": "bearer",
  "expires_in": 3600,
  "user": {
    "id": 1,
    "email": "student@example.com",
    "role": "student",
    "full_name": "John Doe",
    "is_active": true,
    "created_at": "2025-01-15T10:30:00Z"
  }
}
```

Actual Output

```
{
  "access_token":
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "refresh_token":
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "token_type": "bearer",
  "expires_in": 3600,
  "user": {
    "id": 1,
    "email": "student@example.com",
    "role": "student",
    "full_name": "John Doe",
    "is_active": true,
    "created_at": "2025-01-15T10:30:00Z"
  }
}
```

APIs Tests

/api/chatbot/chat

- **Method:** POST
- **Test Case:** Test that chat endpoint requires authentication and return chat message.

```
@pytest.mark.api
@pytest.mark.chatbot
class TestChat:
    """Tests for POST /api/chatbot/chat endpoint."""

    def test_chat_requires_authentication(self, client: TestClient):
        """Test that chat endpoint requires authentication."""
        response = client.post(
            "/api/chatbot/chat",
            json={
                "message": "Hello",
                "mode": "general"
            }
        )

        assert response.status_code == 401
```

Input

```
{
  "message": "Hello",
  "mode": "general"
}
```

Expected Output

```
{
  "detail": "Not authenticated"
}
```

Actual Output

```
{
  "detail": "Not authenticated"
}
```

APIs Tests

/api/chatbot/chat

- **Method:** POST
- **Test Case:** Test that chat endpoint requires authentication and return chat message.

```
@pytest.mark.asyncio
async def test_generate_slide_deck_as_ta(
    client: TestClient, ta_auth_headers: dict, test_course: Course, monkeypatch
):
    """Tests that a TA can successfully generate and save a slide deck."""
    mock_generate = AsyncMock(return_value=MOCK_SLIDE_CONTENT)
    monkeypatch.setattr("app.api.slide_deck_router.slide_deck_service.generate_slides", mock_generate)

    request_data = {
        "course_id": test_course.id,
        "title": "New AI-Generated Slides",
        "description": "Slides about Python basics.",
        "topics": ["Python syntax", "Data types"],
        "num_slides": 2,
    }

    response = client.post("/api/slide-decks/", headers=ta_auth_headers, json=request_data)

    assert response.status_code == 201
    data = response.json()
    assert data["title"] == "New AI-Generated Slides"
    assert data["course_id"] == test_course.id
    assert data["slides"] == MOCK_SLIDE_CONTENT["slides"]
    assert data["creator"]["id"] is not None
    mock_generate.assert_called_once()
```

Input

```
{
  "course_id": <test_course.id>,
  "title": "New AI-Generated
Slides",
  "description": "Slides about
Python basics.",
  "topics": ["Python syntax",
"Data types"],
  "num_slides": 2
}
```

APIs Tests

Expected Output

```
{
  "id": "<generated_id>",
  "title": "New AI-Generated Slides",
  "description": "Slides about Python
basics.",
  "course_id": <test_course.id>,
  "slides": [
    { "title": "...", "content": "..." },
    ...
  ],
  "creator": {
    "id": "<user_id>",
    ...
  },
  "created_at": "<timestamp>"
}
```

Actual Output

```
{
  "id": "deck_982735",
  "title": "New AI-Generated Slides",
  "description": "Slides about Python basics.",
  "course_id": 101,
  "slides": [
    { "title": "Slide 1", "content": "Intro to Python"
  },
    { "title": "Slide 2", "content": "Basic Data
Types" }
  ],
  "creator": {
    "id": 55,
    "role": "teaching_assistant",
    "name": "Test TA"
  },
  "created_at": "2025-01-15T15:32:40.812Z"
}
```

Function Tests

Function/Service : TestSlidesService

Description:

This test suite validates the functionality of the `SlidesService` responsible for generating slide decks from text instructions. The tests focus on **correct Markdown slide generation** and ensure that the output respects the requested theme and instructions.

```
@pytest.mark.asyncio
class TestSlidesService:
    """Groups tests for the SlidesService."""

    @pytest.mark.parametrize("theme", [SlideTheme.MINIMAL, SlideTheme.CORPORATE])
    async def test_generate_slides_success(self, slides_service: SlidesService, theme: SlideTheme):
        """
        Tests successful slide generation for different themes.
        Verifies the structure of the generated Markdown.
        """
        instructions = "Create a 3-slide presentation on the basics of FastAPI.\
            Include a title slide, one content slide about its key features,\
            and a summary slide."

        print(f"\n[TEST] Generating slides with theme '{theme.value}'...")
        markdown_output = await slides_service.generate_slides(
            instructions=instructions,
            theme=theme
        )

        # Assertions
        assert isinstance(markdown_output, str)
        assert "## Error" not in markdown_output, f"API returned an error: {markdown_output}"
        assert "---" in markdown_output, "Slide separator '---' must be present."
        assert markdown_output.strip().startswith("#"), "Output should start with a title slide (#)."
        assert "fastapi" in markdown_output.lower(), "The content should be about FastAPI."

        print(f"[SUCCESS] Slides for theme '{theme.value}' generated and validated.")
        print("=====")
        print(markdown_output)
        print("=====")
```

Input

"Create a 3-slide presentation on the basics of FastAPI.
Include a title slide, one content slide about its key features, and a summary slide."

Expected Output

```
# FastAPI Basics
Subtitle or course name

---

## Key Features of FastAPI
- Very fast (Starlette + Pydantic)
- Auto-generated docs
- Async support

---

## Summary
- Lightweight and async-friendly
- API docs included
- Excellent for microservices
```



Failed Tests

/api/courses/{test_course.id}

Description:

The request payload attempts to change the name of the `test_course` to be the same as the name of `course2` (a duplicate name).

```
def test_update_course_to_duplicate_name_fails(
    client: TestClient, admin_auth_headers: dict, db_session: Session, test_course: Course
):
    """
    Tests that updating a course name to an existing name of another course fails.
    The current implementation will likely raise a 500 error instead of a 409.
    """
    # Create a second course
    course2 = Course(
        name="Second Course Name", description="Another course.", created_by_id=test_course.created_by_id
    )
    db_session.add(course2)
    db_session.commit()

    # Try to update test_course to have the same name as course2
    response = client.put(
        f"/api/courses/{test_course.id}",
        headers=admin_auth_headers,
        json={"name": course2.name},
    )

    # This assertion is expected to fail. The API should return 409, but it will likely return 500.
    assert response.status_code == 409, f"API returned {response.status_code} instead of 409. Body: {response.text}"
    assert "course with this name already exists" in response.json()["detail"]
```

Input

```
{
  "name": "Second Course Name"
}
```

Expected Output

```
{
  "detail": "course with this name already exists"
}
```

Actual Output

```
{
  "detail": "Internal Server Error"
}
```

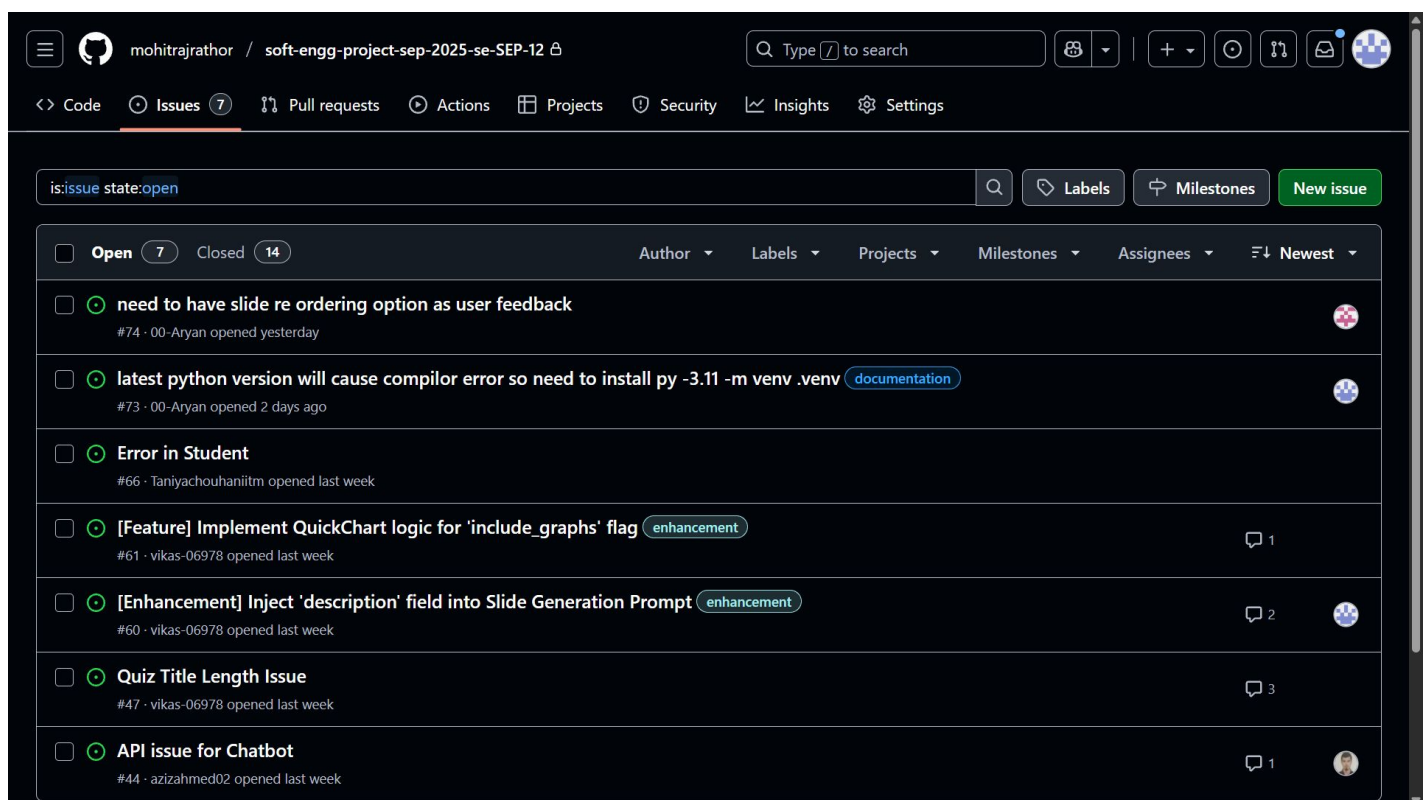
ISSUE REPORTING AND TRACKING

ISSUE REPORTING AND TRACKING

To ensure efficient collaboration and maintain clear oversight of development progress, the team used GitHub Issues as the primary platform for reporting and tracking bugs, feature requests, and technical tasks. GitHub Issues provided a centralized and structured environment where team members could log problems, document steps to reproduce them, and assign responsibility for resolutions. Labels, milestones, and comments were utilized to categorize issues, track their status, and facilitate discussion among team members.

This system allowed the team to manage work asynchronously, maintain transparency in development workflows, and ensure that no issue was overlooked. Throughout the project, GitHub's built-in tools—such as issue linking, notifications, and automated updates via commits and pull requests—helped streamline both communication and progress monitoring. Representative screenshots of the GitHub Issues board have been included below to illustrate how the team organized and tracked ongoing tasks.

OPEN ISSUES



ISSUE REPORTING AND TRACKING

CLOSED ISSUES

mohitrajrathor / soft-engg-project-sep-2025-se-SEP-12

Type [7] to search

<> Code

Issues 7

Pull requests

Actions

Projects

Security

Insights

Settings

is:issue state:closed

Labels

Milestones

New issue

☐ Open 7

☒ Closed 14

Author ▾ Labels ▾ Projects ▾ Milestones ▾ Assignees ▾ Newest ▾

☐ ☒ UI adjustments needed in ta/doubt-summarizer enhancement #72 · by mohitrajrathor was closed 2 days ago 1

☐ ☒ Implement Role based access on frontend #71 · by mohitrajrathor was closed 2 days ago 1

☐ ☒ [Bug] Client-side timeout (10s) too short for Slide Generation bug #63 · by vikas-06978 was closed 2 days ago 1

☐ ☒ In chatboat conversation id is not being generated #55 · by vikas-06978 was closed last week 2

☐ ☒ Error while generating the quizzes #42 · by 00-Aryan was closed last week 1

☐ ☒ Frontend and backend integration bug help wanted #31 · by SAM8402 was closed last week 1

☐ ☒ test new apis enhancement #30 · by SAM8402 was closed last week 1

☐ ☒ Frontend problems bug #27 · by SAM8402 was closed last week 1

☐ ☒ API test question #26 · by SAM8402 was closed last week 1

☐ ☒ remove unnassary jargons from documentation and do formatting documentation enhancement #24 · by mohitrajrathor was closed 3 weeks ago 1

mohitrajrathor / soft-engg-project-sep-2025-se-SEP-12

Type [7] to search

<> Code

Issues 7

Pull requests

Actions

Projects

Security

Insights

Settings

[Bug] Client-side timeout (10s) too short for Slide Generation #63

Edit

New issue

☒ Closed

vikas-06978 opened last week

The frontend Axios instance has a default timeout of 10 seconds (10000ms). Generating a slide deck with Gemini takes longer than 10 seconds, causing the frontend to abort the request with the error: timeout of 10000ms exceeded. Fix Required: Increase the timeout for the /slide-decks endpoint or globally in api/axios.js to at least 60 seconds.

Issue

1. Go to the Slide Generator page.

2. Set "Number of Slides" to 10 (or any number that takes >10s).

3. Click Generate.

4. Result: The error "timeout of 10000ms exceeded" appears exactly at the 10-second mark.

Create sub-issue

vikas-06978 added bug last week

00-Aryan 2 days ago

timeout issue is resolved

00-Aryan closed this as completed 2 days ago

Assignees

No one - Assign yourself

Labels

bug

Projects

No projects

Milestone

No milestone

Relationships

None yet

Development

Code with agent mode

Create a branch for this issue or link a pull request.

Notifications

Customize

Subscribe

You're not receiving notifications from this thread.

Participants

 Milestone 6
TEAM 12

67

TECHNOLOGY USED

TECHNOLOGY USED

BACKEND

fastapi - The modern, high-performance web framework used to build the API endpoints.

uvicorn - The lightning-fast ASGI server used to run the FastAPI application.

pydantic - Used by FastAPI for defining data models, automatic input validation, and serialization/deserialization.

SQLAlchemy - The powerful Python library used as the Object-Relational Mapper (ORM) for database interactions.

alembic - The dedicated tool for managing and running database schema migrations.

Authlib - A comprehensive library for handling OAuth 2.0 and OpenID Connect for user authentication.

PyJWT - A library for encoding and decoding JSON Web Tokens (JWTs), essential for secure API calls.

GENERATIVE AI

langchain - The primary framework for developing applications powered by LLMs.

langgraph - An extension for building stateful, multi-step LLM applications (agents/chains).

google-adk - The Python SDK for directly interacting with Google's Gemini models.

langchain-google-genai - The integration package connecting LangChain to Google's GenAI services.

chromadb - The vector database used for storing and searching vector embeddings (key for RAG).

tokenizers - A high-performance library for tokenizing text, essential for working with LLMs.

FRONTEND

vue - The core Progressive JavaScript Framework used for building the user interface.

pinia - The recommended State Management Library for Vue, used to manage application-wide data.

axios - A popular HTTP client used to make requests to the backend API.

tailwindcss - The utility-first CSS framework used for fast, custom styling.

chart.js - A fundamental JavaScript library for creating various types of charts and graphs.

katex - A fast JavaScript library for rendering complex math equations (LaTeX format) on the web.

vite - The modern, fast build tool and development server for the project.

INSTRUCTION TO RUN THE APPLICATION

INSTRUCTION TO RUN THE APPLICATION

SETUP BACKEND

Unzip the submitted code archive, then follow the instructions.

```
# Move to the backend directory under the project root
cd backend

# Set up a virtual environment with Python 3.11
# This project is built on Python 3.11, so using another version
# may cause issues
python -m venv .venv

# Activate the virtual environment
# On Windows:
.venv\Scripts\activate

# On Linux/macOS:
source .venv/bin/activate

# Install dependencies
pip install -r requirements.txt

# Populate the database with dummy data for testing (optional)
# Skip this step if running in production
python populate_db.py

# Start the application
python main.py
```

SETUP FRONTEND

follow the instructions.

```
# Move to the frontend directory
# Then install dependencies (make sure you have Node.js
# installed)
npm install

# Run the application in development mode
npm run dev

# Build the application for production
npm run build

# (Optional) Run the production build with a suitable server,
# e.g.:
# npx serve dist
```